

Particle Flow Filter and Differentiable Particle Filter

Haowu Duan

March 7, 2026

Contents

1	The big picture	2
1.1	State-Space Modeling in Financial Markets	2
1.1.1	Filtering	2
1.1.2	Beysian inference	2
2	Filters	4
2.1	The Kalman Family and semi-exact methods	4
2.2	Particle based methods	4
2.2.1	Flow and kernal flow	5
2.2.2	kernel Based methods	6
2.2.3	Optimal transport resampling	6
2.3	Hamiltonian Monte Carlo	7
2.3.1	Connection and difference with SGD	9
2.3.2	Implications for differentiable particle filters	9
3	Problems and experiments	10
3.1	State Space models	10
3.2	Code organization and testing strategy	12
3.2.1	Configuration management	12
3.2.2	Code base organization	12
3.2.3	Testing	13
3.3	Part I	14
3.3.1	Kalman-Joseph update-condition number	14
3.3.2	KF, EKF, UKF broken by strong nonlinear model	14
3.3.3	Particle filter and particle degeneracy	15
3.3.4	EDH (LEDH) flow and filter	15
3.3.5	Matrix kernel preventing collapse	17
3.4	Filter comparison	17
3.5	Part II	18
3.5.1	Stochastic flow	18
3.5.2	Soft resampling and OT resampling	20
4	Differentiable particle filter	21
4.1	Filtering the Kitagawa model	21
4.2	PMCMC: gradient-free baseline	22
4.3	SGD-based parameter estimation	23
4.4	The differentiability-bias trade-off	24
4.5	Workflow and current status of DPF pipeline	25
4.6	Benchmarking	26
4.6.1	Benchmarking HMC	26
4.6.2	Benchmarking particle filter and different resampling methods	26
4.7	Additional notes and practical considerations	26
A	Kalman Family	28

B	Particle flow and particle filter	32
B.1	Particle flow	32
B.2	Particle flow filter (kernel method)	34
B.2.1	Optimal Transport via Gradient Flow	34
B.2.2	Deriving Optimal Velocity via Kernel Restriction	34
B.2.3	Scalar Kernel	35
B.2.4	Matrix-Valued Kernel	36
C	MCMC Diagnostics	37
C.1	Acceptance Rate	37
C.2	Step Size and Dual-Averaging Adaptation	37
C.3	Effective Sample Size	37
C.4	$\hat{R}$: Potential Scale Reduction Factor	38

1 The big picture

1.1 State-Space Modeling in Financial Markets

Financial systems are driven by latent factors that shape observable market data. Problems such as stochastic volatility [1](#), [what does it even mean by stochastic volatility?](#), limit order book dynamics, or macroeconomic regime shifts are frequently cast as Hidden Markov Models (HMMs), which has discrete states, or continuous state-space models (SSMs). In this document, we focus on continuous state-space models (SSMs).

Suppose we have some financial data [2](#), [cite some source where financial data is represented as a time series](#). $\{z_k\}$, where k is the time step. This is all we have. The real time evolution of economic factor which might be hidden and not observable is denoted as $\{x_k\}$, sometimes referred to ground truth in controlled experiments. As a one-dimensional demonstration, let us first write down a most generic state-space model,

$$\begin{aligned}x_k &= f(x_{k-1}, w_k) \\ z_k &= h(x_k, u_k)\end{aligned}\tag{1}$$

w_k and u_k are mutually and temporally uncorrelated noises. The function f governs state evolution and h maps the states to observations. In practice we observe only z_k , which may have the same or lower dimension than x_k (the latter case is called partial observation and makes inference substantially harder due to information loss). If we have reliable model that actually describe the laws of economics, then we can accurately make predictions about economic phenomena. A model consists two components, functional form and parameters. Both of which are unknown. In practice, what people do is to make ansatz about the functional form of the model, and fit the parameters of the model, until a better ansatz appears, the said ansatz will represent our best understanding of the phenomena. The empirical observation itself is agonistic to models. To make sure the methods we used can make solid inference of the model parameters, we study the model and tests the relevant numerical methods in controlled experiments. This is the focus of this project. We want to generate artificial data from known model and develop robust numerical methods to extract the parameters of the model. Since the models and parameters are known, there is a clear way to gauge successfulness of the methods. The methods we use to make estimation about the hidden states based on observation is known as **filtering**, and the methods for extracting the model parameter is **Bayesian inference**.

1.1.1 Filtering

Having a good model is only half the battle; we also need a reliable implementation. In research, novel filtering algorithms are developed and evaluated on synthetic data where the ground truth is known, allowing clear performance measurement. Given the state evolution f and observation model h , the goal is to estimate the hidden state x_k via the filtering distribution $p(x_k | z_{1:k})$. This is obtained recursively:

$$\begin{aligned}\textbf{Prediction:} \quad p(x_k | z_{1:k-1}) &= \int p(x_k | x_{k-1}) p(x_{k-1} | z_{1:k-1}) dx_{k-1} \\ \textbf{Update:} \quad p(x_k | z_{1:k}) &= \frac{p(z_k | x_k) p(x_k | z_{1:k-1})}{p(z_k | z_{1:k-1})}\end{aligned}\tag{2}$$

In financial applications (and most real-world settings), we do not aim to recover an exact deterministic value of x_k . Instead, the useful quantities are typically the mean m_k and covariance P_k of the filtering distribution. The filtering problem is generally a high-dimensional path integral and is intractable except in special cases (e.g., linear Gaussian SSMs). Different filtering methods (EKF, UKF, particle filters, etc.) use distinct approximations and mathematical frameworks. The key requirement for any successful filtering algorithm is that it produces numerically stable and accurate approximations to $p(x_k | z_{1:k})$ for known model dynamics.

1.1.2 Bayesian inference

Once a reliable filtering algorithm is available, it can be used to estimate the model parameters θ . We retain the functional forms of f and h but treat their parameters as variables to be learned. A

good parameter set $\{\theta\}$ should assign high probability to the observed sequence. This is achieved by maximizing the marginal likelihood $p_\theta(z_{1:T})$, or equivalently minimizing the negative log-likelihood:

$$L_{MLE}(\theta) = - \sum_{k=1}^T \log p_\theta(z_k \mid z_{1:k-1}) \quad (3)$$

Since the logarithm is monotonically increasing, minimizing $L(\theta)$ maximizes the predictive density of each observation given past data [?]. We denote the optimization to achieve minimum log likelihood as MLE (Maximum log-likelihood estimation). In a Bayesian setting, where parameters are given a prior $p(\theta)$, which add some hard requirement for the range of parameters, we instead minimize,

$$L_{MAP}(\theta) = - \sum_{k=1}^T \log p_\theta(z_k \mid z_{1:k-1}) - \log p(\theta), \quad (4)$$

which corresponds to maximum a posteriori (MAP) estimation. MAP estimation is more informed than MLE, because sometimes, we know for sure that certain parameter range is unreasonable, which could invalidate the model or causing numerical instability for example.

2 Filters

2.1 The Kalman Family and semi-exact methods

For Linear-Gaussian systems, the Kalman Filter (KF) provides the optimal solution because the product of two Gaussian distribution stays as Gaussian. However, financial data rarely satisfies these constraints. Extensions such as the Extended Kalman Filter (EKF) and Unscented Kalman Filter (UKF) utilize linearization and sigma-point approximations, respectively. Either way, for Gaussian distribution, we only need the mean and variance at time k , m_k and P_k .

$$\begin{aligned} x_k &= f x_{k-1} + w_k, & w_k &\sim \mathcal{N}(0, Q) \\ z_k &= h x_k + v_k, & v_k &\sim \mathcal{N}(0, R) \end{aligned} \quad (5)$$

where f and h are constant in one dimension and matrices in higher dimension. When the kernel becomes non-linear, one essentially expand it around the last input.

$$\begin{aligned} f(x_{k-1}) &\approx \left. \frac{\partial f(x)}{\partial x} \right|_{x_{k-1}} \cdot x_{k-1} = f_k x_{k-1} \\ h(x_{k-1}) &\approx \left. \frac{\partial h(x)}{\partial x} \right|_{x_k} \cdot x_k = h_k x_k \end{aligned} \quad (6)$$

essentially introduce time dependence into the kernels but do not change the Gaussian assumption. This works fine if the higher order term are unimportant. UKF use sigma point to track the mean and variance and was able to capture some higher order correlations but it still assume Gaussian distribution. Appendix. A includes all the details such as key derivation. Kalman family mostly serves as a bench mark and have very limited application. Both EKF and UKF are methods to overcome the non-linearity in the problem, with more recent progress in this direction [?]. However, there are other difficulties as well, notably the curse of dimension. Attempts has been made to improve in this direction, notably Cubature Kalman Filter (CKF)[?] and Ensemble Kalman Filter (EnKF)[?, ?]. But here I do want to mention the latest progress on application of nerval network to kalman filter, known as KalmanNet[?, ?]. This is a method that replaces the Kalman Gain computation with a learned neural network while keeping everything else from the standard KF flow intact. The key is to train the RNN to make predictions about the Kalman gain without knowing what the noise distribution is. It gets the noise from the input data. While KalmanNet borrowed design from Kalman filter, it can be applied to non-linear, non-gaussian state space models.

2.2 Particle based methods

Particle based methods, also known as sequential Monte Carlo are the focus of this project. It is less limited compared to other approaches. There are tons of different particle-based methods, but the bottom line is how to represent a complicated distribution efficiently. Here we first introduce the concept of importance sampling. If we were to use particles, namely, samples to represent a distribution $p(x)$, we first need to discretize the phase space.

$$p(x) = \int dy p(y) \delta(y - x) \quad (7)$$

discretized version is,

$$p(x) \propto \sum_i \Delta p(x_i) \delta_{x, x_i} = \sum_i \omega_i \delta_{x, x_i} \quad (8)$$

where we have evenly sampled the phase space points and give them weights the realizes the actual distribution. To put this in the context of sequential Monte Carlo, we demonstrate the so-called Bootstrap particle filter (BPF), at step k , we have the realization of the distribution $\{x_i^{(k-1)}, \omega_i^{(k-1)}\} \sim p(x_{k-1} | z_{1:k-1})$ as input. In the evolution/prediction step,

For $i = 1$ to N :

$$\begin{aligned} x_k^{(i)} &\sim p(x_k | x_{k-1}^{(i)}) \\ \tilde{w}_k^{(i)} &= w_{k-1}^{(i)} \quad (\text{weights unchanged in basic prediction}) \end{aligned} \quad (9)$$

In the above step, the particle simply moved based on physics but their weight stays the same. But we need to use the observation data to score each update

For $i = 1$ to N :

$$\begin{aligned}\tilde{w}_k^{(i)} &= w_{k-1}^{(i)} \cdot p(z_k | x_k^{(i)}) \quad (\text{unnormalized importance weight}) \\ w_k^{(i)} &= \frac{\tilde{w}_k^{(i)}}{\sum_{j=1}^N \tilde{w}_k^{(j)}} \quad (\text{normalized weight})\end{aligned}\tag{10}$$

The major problem with this simple approach is that $p(z_k | x^i)$ are most likely to be a small number. After few steps, most of weights will be so small due to multiplication of multiple small number that they becomes unimportant. The direct consequence is that effective sample size of the distribution becomes small and our discretized representation becomes even coarse grained. This problem is known as particle weight collapse or particle degeneracy. The most straightforward way to deal with this is resampling, which is to create an different realization for the same distribution with more balanced particle sample. Resample particles with replacement according to weights to avoid degeneracy (also called particle depletion or weight collapse). High-weight particles get duplicated; low-weight particles die out. This step is typically performed when $N_{\text{eff}} < N_{\text{threshold}}$ (e.g., $N_{\text{threshold}} = N/2$). The most straightforward resampling method is Multinomial resampling, namely, generate N independent samples from the categorical distribution:

$$\mathbf{x}_t^{(i)} \sim \text{Categorical}(\{w_t^{(j)}\}_{j=1}^N), \quad i = 1, \dots, N\tag{11}$$

where particle $\mathbf{x}_t^{(j)}$ is selected with probability $w_t^{(j)}$. After resampling, set all weights to $w_t^{(i)} = 1/N$ for $i = 1, \dots, N$. In the code, we will use systematic resampling, which is more efficient. For each sample we generate a single uniform random number $u \sim \mathcal{U}(0, 1/N)$ and define:

$$u^{(i)} = u + \frac{i-1}{N}, \quad i = 1, \dots, N\tag{12}$$

Then select particle indices j_i such that:

$$\sum_{k=1}^{j_i-1} w_t^{(k)} < u^{(i)} \leq \sum_{k=1}^{j_i} w_t^{(k)}\tag{13}$$

Set $\mathbf{x}_t^{(i)} = \mathbf{x}_t^{(j_i)}$ and $w_t^{(i)} = 1/N$ for all i . The particles relying on the duplicates or multiplicity to mimic the underlying distribution. As a consequence, we have sample impoverishment problem. Namely, to have more particles with non-trivial weights, we have to duplicate large weight particles while kill the small weight particles. After resampling, we set all particle having equal weight and continues the process.

Because all particles having equal weight, this means the weights carrying no information about the distribution and we rely on multiplicity to represent the distribution right after each resampling. This is extremely inefficient and hint that use multiplicity should've been the our first choice. Then the problem is very similar to numerical procedure to solve stochastic differential equations. Therefore, the prediction step stays the same because it doesn't touch the distribution, just change the basis, it is the update step that responsible to correct the particles. Because there is no diverse weight anymore, the only way to do this is to move the particles again. This is what particle flow particle filter does. These are observation guided evolution equations. When it comes to how, one has a good deal of freedom.

2.2.1 Flow and kernel flow

Alternatively, we don't use weights to represent the distribution at all, namely setting the weight to be uniform. In the update step, we are still using the dynamics to evolve the particle, but in the update step, instead, we move the particles again based on the observation so that the "corrected" distribution is a better realization of the target probability. We will discuss four different ways to move particles.

1. **Exact Flow:** Exact Daum-Huang (EDH) flow and Local Exact Daum-Huang (LEDH) flow[?, ?, ?] are using deterministic PDE to move the particle from prior to posterior. In practice, the algorithms EKF or UKF to move the filter forward. However, this is not the same as assuming the distribution is Gaussian. If the particles is sampled from non-Gaussian distribution, then after Gaussian update, the distribution will still be non-Gaussian. The upside of gaussian assumption is that we avoided computing score $\nabla \ln p$. But the downside is that strong nonlinear problem will break the flow.

2. **Stochastic Flow** [?] Conceptually, the most straightforward way to generalize ODE is SDE. We still make Gaussian assumption about the flow kernel but use stochastic differential equations to do the evolution. When it comes to stiff equations, the paper [?] proposed to use a differential equation to determine the optimal flow schedule instead of straightline.
3. **Invertible Particle flow-Particle filter:** Particle flow equations have discretizations errors, and they don't really transform the particle to the target probability. Think of the flow as a coordinate transformation, not only you should transform the map but also have Jacobian. PF-PF re-introduce the weights back into the picture and use the weight update to make adjustment to compensate the effect of this Jacobian. Because Jacobian is flow specific, if the flow equation kernel is identical across particles, in the case of EDH, then this Jacobian will not contribution and will be canceled by normalization. In the case of particle specific evolution, this Jacobian is highly non-trivial and will be important.

All above methods, on the implementation side, share one important feature. They use EKF/UKF to guide the flow. In appendix. B.1, we have shown how to use Gaussian approximation to derive a closed form solution to the evolution kernel but left out this important implementation details. All particles filters track the mean and empirical variances, and these are the real quantities that reflect the underlying state and uncertainty. Follow up works by different author might include their own custom adjustment. One confusing point I found is the interplay of empirical mean/covariance and EKF/UKF mean and covariance. To derive the evolution kernel, one don't need the parallel EKF to track the mean and variance at all, and this is what early implementation of EDH flow does. Later the authors started running a parallel EKF/UEK filter to track a less noisy mean and covariance. In [?], the author proposed the feedback mechanism to use the empirical mean but EKF covariance to determine the evolution kernel. At the end of the day, there is no one-fits-all solution. Most of the proposed fix are empirical adjustment. Given the underlying ground truth mean, one would like to stay close to the ground truth as possible, the gist of the fix is that one have to try to compare which mean is closer. Though, for highly non-linear system, empirical mean is most likely to be more reliable than EKF mean, given enough particle. In addition, the author of [?] claimed that they took their equation (10) and equation (11) from [?] for linearization performed at intermediate flow time. This is probably a typo, as I found no such equation in [?].

2.2.2 kernel Based methods

kernel-embedded particle flow filter in an RK:[?, ?]: Instead of moving particles with certain schedule, we move the particle along the trajectory where the difference between the particle distribution with the target is decreasing as fast as possible. Define target posterior $\pi(x) := p(x_t|z_{1:t})$ and intermediate density $q_\lambda(x)$ at pseudo-time $\lambda \in [0, 1]$. With $q_0 = p(x_t|z_{1:t-1})$ (prior), $q_1 = \pi$ (posterior).

$$\frac{dx_\lambda}{d\lambda} = v_\lambda(x_\lambda) \quad (14)$$

Choose velocity v_λ to minimize KL divergence as fast as possible:

$$D_{KL}(q_\lambda \parallel \pi) = \int q_\lambda(x) \log \frac{q_\lambda(x)}{\pi(x)} dx \quad (15)$$

we need to formulate the above equation into an optimization problem. But the intuition of this method is clear, you want to evolve the point along a "straight" direction to the desire point. On the implementation side.

2.2.3 Optimal transport resampling

Previously, we have discussed the sample impoverishment problem introduced by low quality resampling. Therefore, we need to think about how to improve it. Before and after resampling, we are preserving the distribution or at latest trying to. We need to come up with the best way to do it. In[?], the author proposed to use Optimal transport to do the resampling. A easy way to see this is that, before the resampling, we have $\{\omega_i, x_i\}$, and after the resampling, we have $\{\tilde{\omega}_i, \tilde{x}_i\}$. If we think x_i are basis vector and ω_i are the coefficient, we are dealing with a linear transformation problem. The transformation will preserve the vector itself, or preserve the mean at least. One should not take this analogy too far. The key benefit is that OT resampling is deterministic instead of stochastic, this will avoid the

sampling noise introduced by systematic resampling therefore removing its contribution to the variance of the particle, namely increasing the quality of covariance estimation. But this method limit the use of neural network architecture in the filtering algorithm. This method is not differentiable because it relies on solving a discrete Linear Programming (LP) problem to find the optimal transport plan. One just uses LP solvers like the Simplex algorithm or Interior Point methods. These algorithms don't need derivatives of the optimal solution with respect to inputs—they only need the cost matrix (particle distances) and constraints (matching weights). They work by walking along edges of the poly-tope or navigating through its interior. The key innovation of [?] is to introduce an entropy regularization so that the optimal coupling P^* is differentiable as function of the cost matrix therefore turning the whole resampling operation differentiable. The following is the loss function for the transport cost,

$$W_{2,\epsilon}^2(\alpha_N, \beta_N) = \min_{P \in S(a,b)} \sum_{i,j=1}^N p_{i,j} \left(c_{i,j} + \epsilon \log \frac{p_{i,j}}{a_i b_j} \right) \quad (16)$$

Another key innovation is that by adding the entropy term $\epsilon \log \frac{p_{i,j}}{a_i b_j}$ instead of $\epsilon \log p_{i,j}$, the author made it possible to determine the transport plan from the result of the dual problem, which is $\mathcal{O}(N)$, instead of $\mathcal{O}(N^2)$ like[?], which open the possibility for modeling more complicated system with large number of particles.

More recently, people also used neural network to solve the Monge problem itself [?]. GradNetOT exploits Brenier's theorem, which says that under certain conditions (continuous densities, squared Euclidean cost), the Monge and Kantorovich problems are equivalent, and the unique transport map $T(x)$ is the gradient of a convex function $\phi(x)$, where $\phi(x)$ satisfies the Monge-Ampère equation

$$p(x) = q(\nabla \phi(x)) \det(H_\phi(x)) \quad (17)$$

where $H_\phi = J_T$ is the $N \times N$ positive semi-definite Hessian of ϕ . But this architecture is not directly applicable to resampling because the transformation will push the continuous distribution $p(x)$ to the target distribution $q(y)$. We will discuss the possible way to make it work.

In addition, there are of course alternatives to OT resampling. Notable example is Particle Transformer[?, ?], where we replaces the OT resampler with a Transformer mechanism (self-attention) to interact particles with each other and re-weight them differentially. To make this review more focused, we will not discuss it further. The key idea is that you demand the transformer to output particles with equal weights. Therefore, it learn from the data to put particles into the right positions.

Optimal transport resampling out-perform in certain scenarios but it's real power lies in that it makes it possible for entire filtering architecture differentiable. Because one can easily use neural network to model the dynamics or observation, which has been extensively discussed in [?]. I think there is no point to make comprehensive review here as it is problem specific and require careful testing, there is no one-fits-all solution. This field is exploding and there are so many different architectures that one can try. As pointed out in the Bonus question 2, it will be nice that we don't have to solve the optimal transport problem repeatedly, because each resampling is a different set of initial particle and weights. One way to deal with this problem is amortized optimization[?], where you train a model to predict solutions to optimization problems rather than solving each problem from scratch. It's application to OT problem is known as Meta-OT [?]. The idea is actually very easy, when we run a filtering algorithm, we need to solve the resampling a lot of times. Each time, we will have a transport schedule, from the dual problem. We simply run filter a lot of times and record what kind of transport plan we will run into in such problems. Then we train neural network to make predictions.

For the purpose of building a differentiable particle filter, we start with entropy-regularized-OT-resampling. It is one more step on top of our regular filter while fully differentiable. This is a good place for benchmarking and ensure the entire pipeline works. Any further improvement or innovative algorithm, namely neural network based algorithm are fully differentiable from the beginning. But they require additional training, for example Meta-OT. As for neural operator based approach, it will also require additional training for solving the Monge-Ampère equation. The idea is to work in parallel, first build the differentiable particle filter pipeline with model-blinded entropy-regularized-OT-resampling. Then for specific model, we can improve on the existing architecture with all the logistic components are ready.

2.3 Hamiltonian Monte Carlo

The whole point of using optimal transport, specifically its differentiable implementation, is to make sure the entire process is differentiable. To make our discussion more focused, our filter will be the

invertible particle flow particle filter [?]. Without a differentiable resampling algorithm, we are forced to use Markov Chain Monte Carlo [?]. The idea is very simple, remember our goal is to find the best parameter set θ , such that negative log-likelihood is minimized. If the system is not fully differentiable, then we can use random walk to propose moves in the parameter space $\theta_{i+1} = \theta_i + w$, where w is a random vector in the parameter space. As a result, we will get a new loss function $L(\theta_{i+1})$, we simply compute the difference $L(\theta_{i+1}) - L(\theta_i) = \Delta L$, if $\Delta L < 0$, then we should accept the move. It is straightforward to incorporate prior distribution for θ . The question is what to do when $\Delta L > 0$. Simply rejecting all uphill moves would trap the chain at the nearest local minimum. Instead, the Metropolis-Hastings algorithm[?] accepts the proposed move with probability

$$\alpha = \min(1, e^{-\Delta L}) = \min\left(1, \frac{p(\theta_{i+1} \mid z_{1:T})}{p(\theta_i \mid z_{1:T})}\right) \quad (18)$$

where the second equality follows from $\Delta L = -\Delta \log p(\theta \mid z_{1:T})$. Even moves that increase the loss are accepted with probability $e^{-\Delta L}$, which allows the chain to escape shallow local optima. This acceptance rule guarantees detailed balance: if we define the transition probability from state θ_i to θ_j as $T(i \rightarrow j) = q(j|i) \alpha(i \rightarrow j)$, where q is the proposal distribution, then $p(\theta_i)T(i \rightarrow j) = p(\theta_j)T(j \rightarrow i)$. Detailed balance in turn guarantees that the Markov chain converges to the correct target distribution, which in our case is the posterior $p(\theta \mid z_{1:T})$ [?].

The fundamental limitation of this random-walk Metropolis approach is that the proposal w is independent of the local geometry of the posterior. In a d -dimensional parameter space, a randomly chosen direction is almost surely nearly orthogonal to any special direction (such as the gradient or the principal axes of the posterior). To maintain a reasonable acceptance rate, the step size must shrink as $\mathcal{O}(d^{-1/2})$, which means the chain explores the posterior very slowly in high dimensions. This motivates using gradient information to construct informed proposals.

In the case of a fully differentiable pipeline, we can use Hamiltonian Monte Carlo[?, ?] to explore the parameter space. The idea is borrowed from classical mechanics. We introduce an auxiliary momentum variable $\mathbf{p}$ of the same dimension as θ and define the Hamiltonian as the total energy,

$$H(\mathbf{p}, \theta) = \underbrace{\frac{1}{2} \mathbf{p}^\top M^{-1} \mathbf{p}}_{\text{kinetic energy } K(\mathbf{p})} + \underbrace{U(\theta)}_{\text{potential energy}} \quad (19)$$

where $U(\theta) = -\log p(\theta \mid z_{1:T})$ is the negative log-posterior and M is a mass matrix (often set to the identity). The corresponding Hamilton's equations,

$$\frac{d\theta}{dt} = \frac{\partial H}{\partial \mathbf{p}} = M^{-1} \mathbf{p}, \quad \frac{d\mathbf{p}}{dt} = -\frac{\partial H}{\partial \theta} = -\nabla_\theta U(\theta) \quad (20)$$

describe trajectories along which the total energy H is exactly conserved. A good analogy is the Harmonic oscillator system $H(p, x) = \frac{p^2}{2} + \frac{x^2}{2}$. The ball in the potential $\frac{x^2}{2}$ will explore the phase space around the minimum with some kinetic energy.

In practice, we integrate these equations numerically using the *leapfrog* (Störmer-Verlet) integrator: half-step momentum update, full-step position update, half-step momentum update. The leapfrog scheme is symplectic, meaning it preserves phase-space volume and approximately conserves the Hamiltonian up to discretization error.

Each HMC iteration proceeds as follows. First, sample a fresh momentum $\mathbf{p} \sim \mathcal{N}(0, M)$. Then simulate L leapfrog steps starting from $(\theta_0, \mathbf{p}_0)$ to obtain a proposal $(\theta^*, \mathbf{p}^*)$. Finally, accept or reject with the Metropolis criterion,

$$\alpha = \min\left(1, e^{-(H(\mathbf{p}^*, \theta^*) - H(\mathbf{p}_0, \theta_0))}\right) \quad (21)$$

Because the leapfrog integrator approximately conserves H , the energy difference $\Delta H \approx 0$, and the acceptance probability is close to 1 even for proposals that are far from the current point in parameter space. This is the key advantage over random-walk Metropolis: the momentum variable carries the state along the energy contour, allowing large, directed moves through parameter space that would be extremely unlikely under a random walk. The price we pay is that each step requires computing $\nabla_\theta U$, which means differentiating through the entire filtering pipeline.

2.3.1 Connection and difference with SGD

Both Hamiltonian Monte Carlo and stochastic gradient descent use the gradient $\nabla_{\theta}U(\theta)$, but for fundamentally different purposes. SGD performs point estimation: it seeks a single best parameter θ^* (MLE or MAP) by iterating $\theta \leftarrow \theta - \eta \nabla U$. Each step uses a different random seed for the particle filter, making the gradient stochastic, but SGD tolerates this because the noise averages out over hundreds of steps. SGD always moves in the gradient direction, there is no accept/reject step.

HMC, by contrast, performs posterior sampling: it characterizes the full distribution $p(\theta \mid z_{1:T})$, not just the mode. HMC requires a *fixed* random seed for the particle filter so that the energy landscape $U(\theta)$ is a deterministic function, because the leapfrog integrator simulates Hamiltonian dynamics and assumes the gradient is a smooth, deterministic force field. If the landscape changed randomly at each evaluation, the trajectory would see contradictory forces and diverge.

This leads to opposite gradient requirements. SGD needs gradients that are *correct on average* (unbiased), and can tolerate high variance. HMC needs gradients that are *smooth at every point*, and a single gradient cliff (where $|\nabla U|$ is enormous) can cause the leapfrog trajectory to explode. We can try to combine them to get best of them. This will be discussed in the later sections.

2.3.2 Implications for differentiable particle filters

When the filtering pipeline is fully differentiable (via soft or OT resampling), SGD or Adam can efficiently find the MAP point. When full posterior uncertainty is needed, HMC requires smooth gradients, and the choice of resampling strategy and filter type becomes critical. As we will see in Section 2.5, the tension between gradient quality (smoothness) and gradient information (low bias) is what drives the design of the differentiable particle filter pipeline.

3 Problems and experiments

3.1 State Space models

We first list all the models that I have used in the numerical experiments.

1. Linear-Gaussian Model

Dynamics:

$$X_n = F \cdot X_{n-1} + B \cdot V_n, \quad V_n \sim \mathcal{N}(0, I)$$

Observation:

$$Y_n = H \cdot X_n + D \cdot W_n, \quad W_n \sim \mathcal{N}(0, I)$$

Parameters:

- $F \in \mathbb{R}^{n_x \times n_x}$: State transition matrix
- $B \in \mathbb{R}^{n_x \times n_v}$: Process noise matrix
- $H \in \mathbb{R}^{n_y \times n_x}$: Observation matrix
- $D \in \mathbb{R}^{n_y \times n_w}$: Observation noise matrix
- $Q = BB^T$: Process noise covariance
- $R = DD^T$: Observation noise covariance

This is the simplest benchmark model for filtering.

2. Acoustic Tracking Full Model (4 targets, 25 sensors)

Dynamics (4 independent targets):

$$X_{t+1} = FX_t + w_t, \quad w_t \sim \mathcal{N}(0, V)$$

where $F \in \mathbb{R}^{16 \times 16}$ is block diagonal with 4 identical blocks, $X \in \mathbb{R}^{16}$

Observation (additive amplitude):

$$z_s = \sum_{i=1}^4 \frac{\Psi}{r_{s,i}^2 + d_0} + \epsilon_s, \quad \epsilon_s \sim \mathcal{N}(0, \sigma_w^2)$$

Parameters:

- $V_{\text{true}} = \frac{1}{20} \text{diag}(V_{\text{single}}, \dots, V_{\text{single}})$ (4 blocks)
- $V_{\text{filter}} = \text{diag}\left(\begin{bmatrix} 3 & 0 & 0.1 & 0 \\ 0 & 3 & 0 & 0.1 \\ 0.1 & 0 & 0.03 & 0 \\ 0 & 0.1 & 0 & 0.03 \end{bmatrix}, \dots\right)$ (4 blocks)
- $\Psi = 10, d_0 = 0.1, \sigma_w = 0.1$
- Sensors: 5×5 grid on $[0, 40] \times [0, 40]$. Basically evenly spread out in the whole area.

3. Range-Bearing Model

Dynamics:

$$X_t = FX_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q), \quad X = [x, y]^T$$

Observation (nonlinear):

$$\begin{bmatrix} \text{range} \\ \text{bearing} \end{bmatrix} = \begin{bmatrix} \sqrt{(x - x_s)^2 + (y - y_s)^2} \\ \arctan 2(y - y_s, x - x_s) \end{bmatrix} + \begin{bmatrix} v_r \\ v_b \end{bmatrix}$$

where $v_r \sim \mathcal{N}(0, \sigma_r^2), v_b \sim \mathcal{N}(0, \sigma_b^2)$

Parameters:

- F : State transition (default: identity)

- Q : Process noise covariance (default: $0.01I_2$)
- (x_s, y_s) : Sensor position (default: origin)
- $\sigma_r = 0.1$: Range noise std
- $\sigma_b = 0.01$: Bearing noise std (radians)

4. Two-Sensor Bearing-Only Model

Dynamics (static):

$$X_{t+1} = X_t, \quad X = [x, y]^T$$

Observation:

$$\begin{bmatrix} \text{bearing}_1 \\ \text{bearing}_2 \end{bmatrix} = \begin{bmatrix} \arctan 2(y - y_{s,1}, x - x_{s,1}) \\ \arctan 2(y - y_{s,2}, x - x_{s,2}) \end{bmatrix} + \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$$

where $v_i \sim \mathcal{N}(0, \sigma_b^2)$

Parameters:

- Prior: $\mu_0 = [3.0, 5.0]^T, \Sigma_0 = \begin{bmatrix} 1000 & 0 \\ 0 & 2 \end{bmatrix}$ (highly anisotropic)
- Sensors: $[(3.5, 0), (-3.5, 0)]$
- $\sigma_b = 0.2$: Bearing noise std
- $R = \text{diag}(0.04, 0.04)$

5. Stochastic Volatility Model

Dynamics (linear):

$$x_t = \alpha x_{t-1} + \sigma w_t, \quad w_t \sim \mathcal{N}(0, 1)$$

Observation (nonlinear, non-Gaussian):

$$y_t = \beta \exp(x_t/2) v_t, \quad v_t \sim \mathcal{N}(0, 1)$$

Parameters:

- $\alpha = 0.91$: Persistence parameter
- $\sigma = 1.0$: Volatility of volatility
- $\beta = 0.5$: Scale parameter
- Stationary variance: $\sigma^2/(1 - \alpha^2)$

6. Lorenz 96 Model

Dynamics (chaotic, high-dimensional):

$$\frac{dx_i}{dt} = (x_{i+1} - x_{i-2})x_{i-1} - x_i + F$$

with cyclic boundary conditions, integrated using 4th-order Runge-Kutta

Observation (linear, sparse):

$$Y = HX + \epsilon, \quad \epsilon \sim \mathcal{N}(0, R)$$

where H observes every 4th variable

Parameters:

- $n_x = 1000$: State dimension
- $F = 8.0$: Forcing constant (chaotic regime)
- $\Delta t = 0.05$: Integration time step
- Observation interval: every 20 RK4 steps
- $\sigma_{\text{obs}} = 1.0$: Observation noise std
- $R = I_{n_y}$, where $n_y = 250$ (25% observed)

7. Kitagawa Model(Model from [?])

$$\begin{aligned} X_n &= \frac{X_{n-1}}{2} + 25 \frac{X_{n-1}}{1 + X_{n-1}^2} + 8 \cos(1.2n) + V_n \\ Y_n &= \frac{X_n^2}{20} + W_n \end{aligned} \tag{22}$$

where $X_1 \sim \mathcal{N}(0, 5)$ and $V_n \sim \mathcal{N}(0, \sigma_V^2)$
 $W_n \sim \mathcal{N}(0, \sigma_W^2)$, where $\theta = (\sigma_v, \sigma_W)$ is our parameter. We only need to estimate the noise scale while the observation and dynamics are fixed.

8. Cubic Sensor Model

Dynamics (linear AR(1)):

$$x_t = a x_{t-1} + v_t, \quad v_t \sim \mathcal{N}(0, \sigma_V^2)$$

Observation (mildly nonlinear):

$$y_t = x_t + c x_t^3 + w_t, \quad w_t \sim \mathcal{N}(0, \sigma_W^2)$$

Parameters:

- $a = 0.9$: AR coefficient
- $c = 0.05$: Cubic nonlinearity strength
- $\sigma_V = 1.0$: Process noise std
- $\sigma_W = 1.0$: Observation noise std
- Initial distribution: $x_0 \sim \mathcal{N}(0, 5)$

The observation function $h(x) = x + cx^3$ has Jacobian $1 + 3cx^2 > 0$ everywhere, so it is injective. Setting $c = 0$ recovers the linear-Gaussian model. This makes it a useful stepping stone between the linear-Gaussian benchmark and the more challenging Kitagawa model: nonlinear enough to require a particle filter, but tame enough that debugging the pipeline is not a headache.

3.2 Code organization and testing strategy

3.2.1 Configuration management

We use Hydra to manage the configuration with yaml files. Hydra is a framework that allows flexible management of the configurations. We start with a default base configuration for model and filters. In practice, we can override the default configuration with new configurations easily and the output file will automatically save a copy of the overwritten version of the configuration. In this way, we can easily create new experiments while keep track of the parameters corresponding to individual experiment outcome.

3.2.2 Code base organization

The codebase follows a modular design where models, filters, and experiment infrastructure are kept independent from each other. The motivation is simple: in filtering research, we constantly need to mix and match different filters with different models. If the model logic and filter logic are entangled, adding a new filter or testing an existing one on a new model becomes unnecessarily painful. By separating these concerns, we can, for instance, run all twenty filters on the linear Gaussian model with a one-line config change.

The source code lives under `src/` and is organized into the following modules:

- `core/` defines the abstract interfaces that the rest of the code builds on. `StateSpaceModel` specifies what any model must provide: sampling methods for state transitions and observations, log-probability evaluations for particle filters, and Jacobians for the EKF/UKF family. `Filter` defines a common filtering interface with `predict()`, `update()`, and `filter()` methods. All concrete implementations inherit from these base classes. This means that every model works with every filter without special-casing, and a new model or filter only needs to implement the abstract interface to integrate with the full pipeline.

- **models/** contains the concrete state-space models. We have ten models ranging from the standard 2D linear Gaussian (which serves as our ground truth benchmark) to highly nonlinear ones like stochastic volatility and range-bearing tracking. Each model provides both a NumPy interface (used by Kalman filters) and a TensorFlow interface (used by particle filters and differentiable methods). The dual interface is necessary because Kalman filters operate on matrices directly while particle filters need batch-vectorized tensor operations for GPU acceleration.
- **filters/** is split into **kalman/** and **particle/**. The Kalman side is relatively simple: a standard Kalman filter (with Joseph form for numerical stability), an Extended Kalman Filter, and an Unscented Kalman Filter. The particle side is where the bulk of the research code lives, with eighteen implementations spanning bootstrap particle filters, kernel-based flow filters, the Exact Daum-Huang family (global and local variants), invertible flow filters with importance weighting, and conditional SMC. A shared base class `ParticleFilterBase` handles diagnostic tracking — ESS history, weight distributions, resampling events — so that every particle filter automatically records the quantities we need for debugging and analysis. With filters inherit structure from easier filter, such as stochastic flow filter inherits from Exact-Daum-Huang flow filter, we only have to debug the updated methods instead of the whole class.
- **resampling/** provides three resampling strategies: systematic resampling (the standard low-variance method), soft resampling (a continuous relaxation that avoids hard index selection, useful for differentiable settings), and optimal transport entropy resampling. Each method follows the same interface, so swapping resampling methods is a config-level change.
- **utils/** provides the numerical building blocks that the rest of the code depends on. For example, numerically stable linear algebra and other supporting function that is repeated called by other filters.
- **experiments/** contains the Hydra-based experiment runner. Given a YAML configuration specifying the model, filter, and parameters, it instantiates the objects, runs the filter, generates diagnostic plots, and records timing and memory statistics. The outputs are saved alongside a copy of the resolved configuration, so every experiment result can be traced back to the exact parameters that produced it.
- **DF/** houses the differentiable filtering module for parameter estimation. This includes runners for Hamiltonian Monte Carlo (HMC), Particle Gibbs (PGibbs), and Particle Marginal Metropolis-Hastings (PMMH). The idea is that once we have a working particle filter, we can treat the log-likelihood it computes as a differentiable function of the model parameters and use gradient-based or MCMC methods to fit those parameters to data.

3.2.3 Testing

Every test fixes the random seed, so the output is deterministic. There is no stochasticity to deal with. The anchor for all filter tests is the 2D linear-Gaussian model with a Kalman filter, which gives the exact posterior mean, covariance, and log-likelihood at every time step. We run each particle filter on the same model and same observations, then check that the posterior mean converges to the Kalman solution and the log-likelihood is within tolerance. All three resampling methods (systematic, soft, OT) are tested independently this way. We also check numerical health — no NaN or Inf in weights, ESS never collapsing, particles not degenerating to a single point — and edge cases like running with only 10 particles or with extremely tight observation noise.

The resampling methods are tested in isolation, outside of any filter. For systematic resampling: selection frequency matches the weights, uniform weights produce an exact permutation, and degenerate weights correctly duplicate a single particle. For soft resampling: $\alpha = 1$ reduces to systematic, $\alpha = 0$ gives uniform sampling, and output weights always sum to one. For OT: output weights are finite, and uniform input weights produce near-identity transport. All three methods are stress-tested with extreme weight ratios. The linear algebra utilities and ODE solvers have their own unit tests against known analytical solutions.

3.3 Part I

3.3.1 Kalman-Joseph update-condition number

The first topic we investigate is Joseph update and condition number. The following table is the calculation based on the 2-d linear Gaussian state space model with parameters:

$$\begin{aligned} F &= \begin{pmatrix} 0.95 & 0 \\ 0 & 0.95 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \\ H &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad D = \begin{pmatrix} 10^{-8} & 0 \\ 0 & 10^{-8} \end{pmatrix} \\ \mu_0 &= \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \Sigma_0 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \end{aligned}$$

where the observation noise is of the order 10^{-8} , which led to $R \sim 10^{-16}$.

Table 1: Comparison of Kalman Filter with Joseph Form vs. Standard Update

Metric	Joseph Form	Standard Update
RMSE	9.708×10^{-9}	9.708×10^{-9}
Log-Likelihood	-566.840	-566.840
Performance Metrics		
Wall Time (s)	0.0203	0.0206
Time per Timestep (ms)	0.1016	0.1030
Peak Memory (MB)	0.274	0.273
Memory Increase (MB)	0.438	0.531
Numerical Stability		
Condition Number	1.0 (all timesteps)	∞ (all timesteps)

Small observation noise will lead to numerical instability and extremely large condition number in the updated covariance. In higher dimension, small noise is not the only such scenario but the argument for why Joseph form will restore numerical stability stays the same. Let us first take a look at what small R in 1-d looks like. With,

$$\lim_{R \rightarrow 0} K = \lim_{R \rightarrow 0} \frac{P^- h}{h^2 P^- + R} = \frac{1}{h} \quad (23)$$

With the standard update: $P^+ = (I - Kh)P^- \approx 0$. But with Joseph form,

$$P^+ = (I - Kh)P^-(I - Kh) + K^2 R \sim \frac{R}{h^2} \quad (24)$$

we have see that $\frac{R}{h^2}$ is basically conditional number one because the noise scale is overall factor. This will legitimately restore numerical instability in higher dimension due to $k^2 R \rightarrow K R K^T$. Let $H = I$, $R = \epsilon I$, $P^- = I$. Then $K \approx I$. The condition number for the residue is one.

3.3.2 KF, EKF, UKF broken by strong nonlinear model

We first use the Stochastic Volatility model,

$$x_{t+1} = 0.91x_t + w_t, \quad w_t \sim \mathcal{N}(0, 1) \quad (25)$$

$$y_t = 0.5 \exp(x_t/2) \cdot v_t, \quad v_t \sim \mathcal{N}(0, 1) \quad (26)$$

with the initial condition drawn from $N(0, \frac{\sigma^2}{1-\alpha^2})$, where $\alpha = 0.91, \sigma = 1$. The result shows that these two filter, though designed to handle non-linear problems, are completely broken by the Stochastic Volatility model. This is due to the fact that when trying to linearize the observation model, the noise term made sure that the mean is always zero. Even though we gave the filter an initial kick, it eventually converged to the zero mean. Another issue is that the variance received zero update due to zero h . In addition, I also did experiment on range bearing model,

State Space: $\mathbf{x}_t = [x_t, y_t]^T \in \mathbb{R}^2$ (2D position)

State Evolution:

$$\mathbf{x}_t = F\mathbf{x}_{t-1} + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(0, Q) \quad (27)$$

where $F = I_2$ (constant position model) and $Q = 0.01 \cdot I_2$. The Observation Model is,

$$\mathbf{y}_t = h(\mathbf{x}_t) + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(0, R) \quad (28)$$

where the nonlinear observation function is:

$$h(\mathbf{x}_t) = \begin{bmatrix} \text{range}_t \\ \text{bearing}_t \end{bmatrix} = \begin{bmatrix} \sqrt{(x_t - x_s)^2 + (y_t - y_s)^2} \\ \arctan 2(y_t - y_s, x_t - x_s) \end{bmatrix} \quad (29)$$

Parameters:

- Sensor position: $\mathbf{x}_s = [x_s, y_s]^\top = [0, 0]^\top$
- Observation noise: $R = \text{diag}(\sigma_{\text{range}}^2, \sigma_{\text{bearing}}^2)$
 - Range noise: $\sigma_{\text{range}} = 0.1$
 - Bearing noise: $\sigma_{\text{bearing}} = 0.01$ radians
- Initial state: $\mathbf{x}_0 \sim \mathcal{N}(\boldsymbol{\mu}_0, \Sigma_0)$ where $\boldsymbol{\mu}_0 = [1, 1]^\top$, $\Sigma_0 = I_2$

Jacobian: For the Extended Kalman Filter, the observation Jacobian is:

$$H(\mathbf{x}_t) = \frac{\partial h}{\partial \mathbf{x}} \bigg|_{\mathbf{x}_t} = \begin{bmatrix} \frac{x_t - x_s}{r_t} & \frac{y_t - y_s}{r_t} \\ -\frac{y_t - y_s}{r_t^2} & \frac{x_t - x_s}{r_t^2} \end{bmatrix} \quad (30)$$

where $r_t = \sqrt{(x_t - x_s)^2 + (y_t - y_s)^2}$ is the range.

we see that range bearing model works with UKF and EKF because we can get reasonable Jacobian from the observation model.

3.3.3 Particle filter and particle degeneracy

The red lines are the times where we did resampling. It turns out that range bearing had very bad initialization but the particle filter somehow managed to increased sample size at later time. This is likely due to that particles were moved into the right position so even with very few particles, it can track the distribution very well. But a depleted sample size means that uncertainty estimation is unreliable.

3.3.4 EDH (LEDH) flow and filter

Table 2 compares all five filters on the range-bearing model. The LEDH invertible filter achieves the best RMSE while the bootstrap PF requires the most particles. Table 3 shows results on the acoustic tracking model (4 targets, 25 sensors), a higher-dimensional problem where the global EDH filters struggle due to the linearisation approximation. The LEDH variants, which re-linearise at each particle, significantly outperform their global counterparts, with LEDH invertible again achieving the best RMSE.

Table 2: Particle Filter Comparison on Range-Bearing Model

Filter	RMSE	Log-Lik	Wall Time (s)	Time/Step (ms)	Mem (MB)
PF	0.1135	522.66	98.7	197.4	1684.7
EDH Flow	0.1687	—	93.4	934.2	474.3
EDH Invertible	0.1199	82.85	48.4	967.7	646.7
LEDH Flow	0.1135	—	455.3	910.6	574.3
LEDH Invertible	0.1130	1040.43	474.1	948.1	947.0

Filter	Particles	Resample Rate	Timesteps
PF	1000	0.70	500
EDH Flow	500	—	100
EDH Invertible	500	0.88	50
LEDH Flow	500	—	500
LEDH Invertible	500	0.57	500

Table 3: Particle Filter Comparison on Acoustic Tracking Model (4 targets, 25 sensors)

Filter	RMSE	Log-Lik	Wall Time (s)	Time/Step (ms)	Mem (MB)
PF	23.61	-14158.41	3.3	65.3	101.3
EDH Flow	85.62	—	65.3	1306.5	1568.7
EDH Invertible	67.93	-13029.33	77.9	1557.3	2179.0
LEDH Flow	17.64	—	86.8	1735.4	1770.5
LEDH Invertible	16.68	1048.10	89.6	1791.9	2010.8

Filter	Particles	Resample Rate	Timesteps
PF	1000	1.00	50
EDH Flow	500	—	50
EDH Invertible	500	1.00	50
LEDH Flow	500	—	50
LEDH Invertible	500	1.00	50

Table 4 extends the acoustic tracking experiment to $T = 100$ timesteps and studies the effect of geometric lambda schedules on the LEDH invertible filter. The geometric schedule concentrates integration steps near $\lambda = 0$ (where the posterior changes most rapidly) by spacing steps as $\lambda_k \propto q^k$. Results show that $q = 1.5$ with $n = 37$ steps achieves the best RMSE (31.56), because it runs for longer, so RMSE is expected to be twice as much as $T = 50$ case approximately.

Table 4: LEDH Invertible with Geometric Lambda Schedule on Acoustic Tracking ($T = 100$)

q	n_λ	RMSE	Log-Lik	Wall Time (s)	Time/Step (ms)
1.2	25	54.17	-6972.72	143.4	1433.7
1.2	29	48.54	1806.41	160.5	1605.1
1.2	33	48.78	-6595.54	179.2	1792.1
1.2	37	54.35	-6160.39	198.3	1983.0
1.2	42	52.09	-7219.87	226.6	2266.2
1.5	25	49.18	1835.16	146.6	1465.7
1.5	29	47.83	-6568.38	165.0	1650.3
1.5	33	44.43	1776.00	183.8	1838.0
1.5	37	31.56	988.83	197.5	1975.3
1.5	42	47.14	-6577.46	221.2	2211.5

All runs: 500 particles, resample rate 1.0, float32. Best result in bold.

Table 5 compares filters on the stochastic volatility model, a 1D model with a strongly nonlinear observation $y_t = 0.5 \exp(x_t/2) \cdot v_t$. The EKF and UKF linearise $h(x) = 0.5 \exp(x/2)$ at the predicted mean; when the mean is near zero the Jacobian $h'(x) \approx 0.25$ is small and effectively gives zero information gain, causing them to track the prior rather than the data (RMSE 2.54 vs PF's 1.14). The EDH flow suffers similarly: its global linearisation at $\bar{\eta}_0$ yields $H \approx h'(\bar{\eta}_0)$, which degrades when the prior mean is uninformative. The bootstrap PF avoids linearisation entirely and achieves the best RMSE.

Table 5: Filter Comparison on Stochastic Volatility Model

Filter	RMSE	Log-Lik	Wall Time (s)	Time/Step (ms)	Mem (MB)
PF	1.14	-547.56	2.1	4.2	46.4
EKF	2.54	-8731.11	2.7	5.4	40.6
UKF	2.54	-8731.11	5.2	10.3	51.2
EDH Flow	2.58	—	33.3	66.5	64.1
Stochastic EDH	2.60	—	53.2	106.5	65.9
LEDH Flow*	3.21	—	3.4	168.5	30.3
Kernel (scalar)	2.71	—	224.9	449.9	416.1
Kernel (matrix)	2.71	—	328.7	657.5	548.7

Filter	Particles	Resample Rate	Timesteps
PF	1000	0.36	500
EKF	—	—	500
UKF	—	—	500
EDH Flow	500	—	500
Stochastic EDH	500	—	500
LEDH Flow*	500	—	20
Kernel (scalar)	20	—	500
Kernel (matrix)	20	—	500

*LEDH Flow ran for only $T = 20$ timesteps.

3.3.5 Matrix kernel preventing collapse

3.4 Filter comparison

Overall, there is no one-fits-all filter option. When the model is non-linear but with additive noise, then flow based approach would work with carefully tuned hyper parameters. As you can see, there is a lot of freedom to adjust the flow equation implementation. With multiplicative noise, I cannot say for certain, but I would expect most of the expansion approach would fail because the observation is not useful at all, for flow evolution.

3.5 Part II

3.5.1 Stochastic flow

The stochastic particle flow filter of [?] generalises the deterministic EDH flow from an ODE to a stochastic differential equation:

$$d\mathbf{x} = [A(\lambda)\mathbf{x} + b(\lambda)] d\lambda + \sqrt{Q} dW_\lambda,$$

where $Q = \text{diag}(q_1, \dots, q_d)$ is a diagonal diffusion matrix. The drift includes a score correction (Corollary 2.1 of [?]) so that the posterior is exact in the continuous-time limit for any choice of Q .

The deterministic EDH flow (ODE) can freely re-linearise H at each particle’s position—the local (LEDH) approach—because the ODE drift is simply a velocity field that each particle follows independently. The stochastic flow is fundamentally different. The drift contains a score correction:

$$f_{\text{stoch}}(\mathbf{x}, \lambda) = f_{\text{det}}(\mathbf{x}, \lambda) + \frac{1}{2} Q \nabla_{\mathbf{x}} \log p(\mathbf{x}, \lambda),$$

where $p(\mathbf{x}, \lambda) \propto g(\mathbf{x}) h(\mathbf{x})^\lambda$ is the homotopy density. This correction compensates for the injected diffusion noise so that the particle density stays on the homotopy path. Formally, the drift must satisfy the Fokker–Planck equation

$$\frac{\partial p}{\partial \lambda} = -\nabla \cdot (f p) + \frac{1}{2} \nabla \cdot (Q Q^\top \nabla p),$$

a *global* constraint: there must exist a single density $p(\mathbf{x}, \lambda)$ whose score $\nabla \log p$ appears in the drift for every particle simultaneously.

Under global linearization (H fixed at $\bar{\eta}_0$), $p(\mathbf{x}, \lambda)$ is Gaussian with precision $M(\lambda) = P_0^{-1} + \lambda H^\top R^{-1} H$, and the score

$$\nabla \log p(\mathbf{x}, \lambda) = -M(\lambda)(\mathbf{x} - m(\lambda))$$

is a single linear function of $\mathbf{x}$ shared by all particles—consistent with the Fokker–Planck equation.

If one were to re-linearise at each particle $\mathbf{x}_i$ (as LEDH does for ODE flows), particle i would use its own H_i , giving it its own precision $M_i(\lambda)$, mean $m_i(\lambda)$, and score $-M_i(\mathbf{x} - m_i)$. No single density has all of these as its score simultaneously—the Fokker–Planck equation cannot be satisfied. The particles would no longer be samples from a coherent density, and the theoretical guarantee that the flow preserves the homotopy path breaks down. This is why stochastic particle flows in the literature universally use global linearization, while the local (LEDH) variant is restricted to deterministic flows. Given this fact, in the code base, we can directly inherit the structure of EDH with small override to the flow parameter. For strong non-linear model, we need to add local corrections to the flow parameters, or re-linearize at the intermediate time. But for Stochastic flow, re-linearize is not possible, we cannot update the observation coefficient, but it turns out adding local correction will help stabilize the flow.

The drift parameters $A(\lambda)$ and $b(\lambda)$ are derived under the assumption of a linear observation model $h(\mathbf{x}) = H\mathbf{x}$. In the pure global formulation—where H is computed once at the prior mean $\bar{\eta}_0$ and the observation z is used directly—this linear model is self-consistent and no additional correction is needed.

However, in the re-linearisation framework (as in LEDH), one expands $h(\mathbf{x})$ at an intermediate point η during the flow. The first-order Taylor expansion

$$h(\mathbf{x}) \approx h(\eta) + H(\eta)(\mathbf{x} - \eta) = H(\eta)\mathbf{x} + \underbrace{[h(\eta) - H(\eta)\eta]}_e,$$

naturally produces a zeroth-order residual $e = h(\eta) - H(\eta)\eta$ that accounts for the nonlinearity. In our local correction variant, we borrow this idea for the global formulation: compute $e_0 = h(\bar{\eta}_0) - H\bar{\eta}_0$ once, then replace z with $z_{\text{corr}} = z - e_0$ in both the deterministic drift and the score correction. For linear observations $e_0 = 0$ and the two formulations coincide.

Empirically, this correction is essential for nonlinear models. On the range-bearing model, the uncorrected stochastic EDH diverges, while adding the local correction makes the filter work (Figures ?? and ??). The correction costs only one additional evaluation of $h(\bar{\eta}_0)$ per time step. As for stiffness mitigation, the drift matrix $A(\lambda)$ has eigenvalues that are most extreme near $\lambda = 0$, where the precision $M(\lambda) = P_0^{-1} + \lambda H^\top R^{-1} H$ is dominated by the anisotropic prior P_0^{-1} . As $\lambda \rightarrow 1$, measurement information regularizes M and the stiffness diminishes. With a sufficiently fine uniform grid (small $d\lambda$), the Euler–Maruyama integrator stays within the stability region even near $\lambda = 0$ —a fine uniform schedule is not stressed by the stiff system.

In practice, one can do better with a geometric (exponential) schedule: step sizes $\varepsilon_j = \varepsilon_1 q^{j-1}$ with ratio $q = 1.2$, which concentrates steps near $\lambda = 0$ where they are most needed and uses larger steps near $\lambda = 1$ where the system is well-conditioned. This simple engineering choice already resolves the practical stiffness issue without solving any optimization problem. I spent sometime to test and reproduce the result of the stiffness mitigation paper but could not reproduce. The filter itself works fine, but I find the usefulness of the method proposed by the author is limited, and the stiffness in the example can be mitigated by fine uniform step size. One thing I did not try is to switch the integration scheme. We evaluate our parameter after we update the step counts, follow the pseudocode from [?]. As we know the first step has strong stiffness problem, this could be the reason why I could not reproduce the paper.

3.5.2 Soft resampling and OT resampling

Table 6: Resampling Method Comparison on Stochastic Volatility Model

Method	Param	RMSE	Log-Lik	Wall Time (s)	Time/Step (ms)	Resample Rate
<i>Baseline</i>						
PF (Systematic)	—	1.142	−547.56	2.12	4.25	0.358
<i>Optimal Transport (OT) Resampling</i>						
OT	$\epsilon = 0.1$	1.582	−702.84	359.45	718.91	0.420
OT	$\epsilon = 0.3$	1.560	−839.05	182.84	365.68	0.432
OT	$\epsilon = 0.5$	1.841	−703.65	156.06	312.12	0.428
OT	$\epsilon = 1.0$	2.110	−723.65	47.94	95.89	0.432
<i>Soft Resampling</i>						
Soft	$\alpha = 0.5$	1.140	−545.37	2.32	4.64	0.532
Soft	$\alpha = 0.7$	1.137	−544.05	2.26	4.52	0.426
Soft	$\alpha = 0.9$	1.135	−545.60	2.26	4.52	0.370

PF (Systematic) works really well for Stochastic volatility model for the purpose of doing the filtering. We see that as we increase the ϵ , the effect of smoothing out the particle distribution from OT resampling degrade the filtering result but it takes less time to run. Accuracy, namely less regularization requires more computation. Soft resampling is a modification of multinomial resampling, which is cheap to run, and less regularization is more accurate, consistent with OT resampling as well.

4 Differentiable particle filter

The goal is to build a differentiable particle filter on kitagawa model,

$$\begin{aligned} X_n &= \frac{X_{n-1}}{2} + 25 \frac{X_{n-1}}{1 + X_{n-1}^2} + 8 \cos(1.2n) + V_n \\ Y_n &= \frac{X_n^2}{20} + W_n \end{aligned} \tag{31}$$

where $X_1 \sim \mathcal{N}(0, 5)$ and $V_n \sim \mathcal{N}(0, \sigma_V^2)$. $W_n \sim \mathcal{N}(0, \sigma_W^2)$, where $\theta = (\sigma_V, \sigma_W)$ is our parameter. The key difficulty is the observation model $Y_n = X_n^2/20 + W_n$: squaring the state destroys sign information, making the filtering distribution bimodal at every time step. Both $+x$ and $-x$ produce the same expected observation $x^2/20$. This means standard filters cannot distinguish the two modes from a single observation, and parameter estimation inherits this structural difficulty. We take a three-step approach:

1. **Filtering results.** We first run bootstrap particle filter, LEDH invertible filter on the Kitagawa model to demonstrate that this is a structurally difficult problem. Then we propose an improvement using the observation of the next k step to distinguish the two modes. Even though, it is not always true that most accurate filter is best suitable for Differentiable particle filter, it doesn't hurt to try the improvement then figure out ways to make it compatible with dpf pipeline.
2. **PMCMC paper analysis.** We discuss the approach of Andrieu et al.[?], who applied Particle Gibbs with a conjugate Inverse Gamma θ -step to this exact model. Their method works, but relies on a conjugate shortcut that is not available for general models. We will report our findings in the actual section.
3. **HMC differentiable particle filter pipeline.** We build a full differentiable pipeline, and report our findings so far, including the difficulties such initialization, comparison between different resampling methods, and how do we plan to improve it further.

4.1 Filtering the Kitagawa model

Before attempting parameter estimation, we first establish how well various filters track the latent state when the true parameters are known ($\sigma_V^2 = 10$, $\sigma_W^2 = 1$). Table 7 summarizes the results for $T = 100$ time steps.

Filter	N	RMSE	Log-lik	Time (s)
EKF	—	33.06	−1431.7	1.5
Bootstrap PF	1000	19.66	−360.6	1.4
EDH Invertible	500	20.67	−132.7	18.3
LEDH Invertible	500	19.91	−135.4	15.3
LEDH Flow	500	18.93	—	14.5
Bootstrap PF	10000	19.27	−359.0	1.3
LEDH Invertible	10000	19.50	−137.0	33.2
LEDH Bimodal ($K=1$)	500	11.57	−205.4	18.9

Table 7: Filtering performance on Kitagawa model ($T = 100$, seed=42, float64). Log-likelihood is the particle filter's estimate of $\log p(z_{1:T})$.

Three observations stand out. First, all standard filters (BPF, EDH, LEDH) converge to RMSE ≈ 19 –21 regardless of method. The problem is structural: the observation $y = x^2/20$ provides zero sign information, so every filter tracks the magnitude correctly but guesses the sign at chance ($\approx 50\%$). Second, increasing particles by $20\times$ (from 500 to 10,000) gives less than 1% RMSE improvement, confirming that the bottleneck is not sample size but the bimodal posterior. Third, the LEDH bimodal filter with a 1-step look-ahead correction (using the next observation y_{t+1} to disambiguate sign via the asymmetric transition dynamics $f(x) \neq f(-x)$) breaks through to RMSE 11.57, a 42% improvement. The key insight is that while a single observation y_t cannot distinguish $+x$ from $-x$ (since $h(x) = x^2/20$ is an even

function), the *next* observation y_{t+1} can, indirectly. For each flowed particle η_1 , we propagate both signs one step forward through the transition:

$$x_{t+1}^+ = f(+\eta_1, t+1), \quad x_{t+1}^- = f(-\eta_1, t+1), \quad (32)$$

and compute predicted observations $\hat{y}^\pm = (x_{t+1}^\pm)^2/20$. Because the transition $f(x, t) = x/2 + 25x/(1 + x^2) + 8\cos(1.2t)$ contains the even cosine term $8\cos(1.2t)$, we have $f(+x) \neq -f(-x)$ in general, so the two predicted observations differ. Comparing $\hat{y}^+$ and $\hat{y}^-$ against the actual y_{t+1} tells us which sign is more consistent with the data. The flip probability is

$$P_{flip} = \sigma\left(\log p(y_{t+1} | x_{t+1}^-) - \log p(y_{t+1} | x_{t+1}^+)\right), \quad (33)$$

where the log-likelihoods use a predictive variance that accounts for process noise propagated through the observation Jacobian. For example, at $t = 50$, we have $\cos(1.2 \times 51) \approx -0.95$:

$$\begin{aligned} f(+15) &= 7.5 + 1.66 - 7.6 = 1.56 & \implies & \hat{y}^+ = 1.56^2/20 = 0.12, \\ f(-15) &= -7.5 - 1.66 - 7.6 = -16.76 & \implies & \hat{y}^- = 16.76^2/20 = 14.05. \end{aligned}$$

If $y_{t+1} \approx 14$, the negative mode is overwhelmingly favored (signal difference ~ 14 vs ~ 0.1). Table 8 shows how the look-ahead depth K affects performance.

K	RMSE	Log-lik	Mean ESS	Time (s)
0 (baseline)	19.91	-135.4	~ 200	12.0
1	11.57	-205.4	170.8	18.9
2	12.89	-215.2	211.2	37.1
5	11.80	-196.2	223.5	21.3
7	15.35	-208.4	218.4	22.3
10	17.19	-214.7	233.6	24.0

Table 8: LEDH Bimodal look-ahead depth sweep ($N = 500$, $n_\lambda = 29$, $T = 100$, seed=42, float64).

$K = 1$ is already near-optimal (RMSE 11.57, a 42% improvement over baseline). One step of deterministic propagation through $f(x, t)$ is already asymmetric enough to provide strong discriminative signal. For $K \geq 5$, the deterministic rollout ignores process noise ($\sigma_V = 10$), so later predictions diverge from reality and contribute noise rather than signal to the score. The degradation at $K = 7$ and $K = 10$ confirms this. $K = 1$ is preferred for its simplicity, lowest wall time, and best RMSE. In addition, we note that RMSE and log-likelihood do not follow the same trend. The standard LEDH invertible has the best log-likelihood (-135) despite mediocre RMSE (19.9): all particles concentrate in one mode, producing well-calibrated weights but a biased point estimate. The bimodal filter spreads particles across both modes, improving the point estimate (lower RMSE) but diluting the predictive density (lower log-likelihood). This dissociation is characteristic of bimodal filtering problems.

We keep in mind that **the sign correction is not differentiable**. The look-ahead scoring produces a smooth flip probability $p_{flip} = \sigma(\cdot)$ via the sigmoid, but the actual flip decision uses a hard threshold which means if we wanted to embed this filter inside a differentiable particle filter framework—learning θ by minimizing $-\log p(y_{1:T} | \theta)$ via gradient descent through the entire filter—the sign correction would block gradients. The loss would not see the effect of θ on the sign decisions. In the future, we can try to replace the hard threshold with a soft threshold, or other novel improvement.

4.2 PMCMC: gradient-free baseline

Andrieu et al.[?] introduce two gradient-free MCMC methods that use the particle filter as a black-box likelihood estimator, avoiding all differentiability issues. Particle Marginal Metropolis-Hastings replaces the intractable likelihood $p(z_{1:T}|\theta)$ with the particle filter’s unbiased estimate $\hat{p}(z_{1:T}|\theta)$, then runs standard Metropolis-Hastings:

$$\alpha = \min\left(1, \frac{\hat{p}(z_{1:T}|\theta^*) p(\theta^*)}{\hat{p}(z_{1:T}|\theta^{(i)}) p(\theta^{(i)})}\right) \quad (34)$$

Their Theorem 2 shows this leaves the correct posterior invariant for *any* $N \geq 1$ particles. No gradients are needed, so gradient cliffs and resampling discontinuities are irrelevant. The variance of the likelihood

estimate scales as $\mathcal{O}(T/N)$ (their Theorem 1), so doubling T requires doubling N to maintain the same acceptance rate. In addition, they use particle Gibbs (PG) to split the joint posterior $p(\theta, x_{0:T} \mid z_{1:T})$ via Gibbs sampling, alternating between a θ -step and an x -step. In the θ -step, given a fixed trajectory $x_{0:T}$, the conditional posterior is

$$\log p(\theta \mid x_{0:T}, z_{1:T}) = \sum_{t=1}^T \log p(x_t \mid x_{t-1}, \theta) + \sum_{t=1}^T \log p(z_t \mid x_t, \theta) + \log p(\theta) + \text{const} \quad (35)$$

Each term is a Gaussian log-density. No particle filter is involved — no resampling, no Jacobian accumulation, no weight collapse. For the Kitagawa model with $\theta = (\sigma_V, \sigma_W)$, this becomes

$$\log p(\sigma_V, \sigma_W \mid x_{0:T}, z_{1:T}) = -T \log \sigma_V - \frac{1}{2\sigma_V^2} \sum_{t=1}^T (x_t - f(x_{t-1}, t))^2 - T \log \sigma_W - \frac{1}{2\sigma_W^2} \sum_{t=1}^T \left(z_t - \frac{x_t^2}{20} \right)^2 + \log p(\sigma_V, \sigma_W) \quad (36)$$

where $f(x, t) = x/2 + 25x/(1+x^2) + 8 \cos(1.2t)$. The paper parameterizes in terms of variances σ_V^2, σ_W^2 with Inverse Gamma (IG) priors: $\sigma_V^2 \sim \text{IG}(0.01, 0.01)$. The Gaussian likelihood, viewed as a function of σ^2 , has the form

$$p(\text{data} \mid \sigma^2) \propto (\sigma^2)^{-T/2} \exp\left(-\frac{S}{2\sigma^2}\right), \quad S = \sum_{t=1}^T e_t^2 \quad (37)$$

where $e_t = x_t - f(x_{t-1}, t)$ are the transition residuals. This is IG-shaped in σ^2 . Multiplying by the IG prior preserves the functional form:

$$\sigma_V^2 \mid x_{0:T} \sim \text{IG}\left(\alpha_0 + \frac{T}{2}, \beta_0 + \frac{1}{2} \sum_{t=1}^T (x_t - f(x_{t-1}, t))^2\right) \quad (38)$$

and similarly for σ_W^2 with observation residuals $r_t = z_t - x_t^2/20$. The θ -step reduces to computing a sum of squared residuals and drawing one sample from an IG distribution — no MH, no HMC, 100% acceptance, zero tuning. The paper presents this as simply “sample θ from the conditional” without elaboration, but it is a significant hidden advantage: the θ -step is exact, instant, and requires no hyperparameter choices.

In the x -step, a new trajectory $x_{0:T}$ is sampled via Conditional SMC (CSMC)[?, ?], which pins one particle (the *reference*) to the previous iteration’s trajectory while letting $N - 1$ free particles explore. Reference pinning guarantees ergodicity, and ancestor sampling[?] improves mixing by allowing the reference particle’s history to graft onto other particles’ pasts.

The paper’s successful results use $N = 5000$ particles, $T = 500$ time steps, 50,000 MCMC iterations, and the conjugate IG θ -step. We want to borrow what we can to make the problem easier as a step stone toward full dpf framework. We can split the whole dpf procedure into MC/HMC + CSMC, where the CSMC part can be further improved with the look-ahead flip trick we invented for kitagawa model. Currently, we have done small test run to ensure numerical stability, and we observe that the algorithm is very sensitive to initial guess, with a bad initial guess, the parameters will drift away instead of oscillating around the true value. For production run, we will need to make good guess on the initial guess before we run Bayesian estimation on the model parameter. Which we will discuss in the next section.

4.3 SGD-based parameter estimation

The simplest way to test the differentiable particle filter pipeline is to use gradient-based optimization. Given the differentiable pipeline, we treat the negative log-posterior as a loss function and minimize it with Adam or SGD,

$$\theta \leftarrow \theta - \eta \nabla_{\theta} [-\log p(z_{1:T} \mid \theta) - \log p(\theta)] \quad (39)$$

where $\log p(z_{1:T} \mid \theta)$ is the log marginal likelihood from the particle filter. Each optimization step uses a different random seed for the particle filter, making the gradient stochastic, but SGD handles this naturally by averaging over hundreds of steps. SGD-based estimation finds the MAP point efficiently but provides no uncertainty quantification: it returns a single best estimate θ^* without characterizing how confident we should be in that estimate. Currently, this is implemented in the codebase, but no experiment was done. Eventually, we can add this to the warming-up stage to improve the exploration efficiency.

4.4 The differentiability-bias trade-off

For full posterior estimation (uncertainty quantification), we use HMC on a deterministic likelihood surface by fixing the particle filter seed. This gives us the posterior distribution $p(\theta \mid z_{1:T})$, not just the mode. However, the particle filter’s log-likelihood surface has features that make HMC challenging.

The true gradient $\nabla_{\theta} \log \hat{p}(z_{1:T} \mid \theta)$ should account for how θ affects everything: the state transitions, the observation weights, and the resampling decisions — which particles survive and which are killed. Resampling selects particles by mapping continuous weights to integer indices, a piecewise-constant operation with zero derivative almost everywhere. Autodiff passes through this operation and misses the dependence of particle selection on θ . The *gradient bias* is precisely this missing “selection gradient.” The three resampling strategies handle this differently:

Systematic resampling selects particles by mapping cumulative weights to integer indices — a discrete, non-differentiable operation. If we detach the gradient at the resampling step (treating the selected indices as fixed constants), autodiff captures how θ affects particle values and observation log-probabilities, but learns nothing about which particles were selected. The resulting gradient is heavily biased but well-behaved: it is bounded and varies continuously with θ , because it only differentiates through smooth transition and observation models. However, the loss surface *itself* is not smooth. It is piecewise smooth with discontinuities wherever a small change in θ causes a different particle to be selected. With N particles and T resampling steps, there are up to $\sim NT$ such boundaries.

Soft resampling[?] blends the normalized weights with a uniform distribution, $\tilde{w}_i = \alpha w_i + (1 - \alpha)/N$, making the importance weight correction $w_{\text{new}} = w/\tilde{w}$ differentiable. However, the core particle selection is unchanged — it still maps cumulative weights to integer indices. Autodiff captures the weight correction but not the selection. This makes soft resampling *partially* differentiable with medium gradient bias.

OT Sinkhorn resampling[?] replaces discrete index selection entirely with a continuous transport: $x'_i = \sum_j T_{ij} x_j$, where T is a doubly-stochastic matrix computed via differentiable Sinkhorn iterations. Every input particle contributes to every output particle with a smooth, differentiable weight. This is the only *fully* differentiable resampling method, producing gradients with low bias. The cost is that differentiating through the Sinkhorn iterations can produce large gradient magnitudes — the gradient is correct but potentially unbounded.

Table 9: Resampling strategies for differentiable particle filters.

Resampling	Fully diff.?	Gradient bias	Gradient bounded?	Loss continuous?
Systematic (detached)	No	High	Yes	No
Soft (PF-net)	Partial	Medium	Yes	No
OT Sinkhorn	Yes	Low	Not guaranteed	Yes

HMC’s leapfrog integrator simulates Hamiltonian dynamics and treats the gradient as a deterministic force field. It requires two properties:

1. *Bounded gradients.* A single gradient spike can send the leapfrog trajectory to infinity; the proposal is rejected but no progress is made.
2. *A continuous loss surface.* Discontinuities in the Hamiltonian cause large energy errors ΔH , leading to rejected proposals and step-size collapse.

Systematic resampling with detached gradients satisfies (1) but not (2): the biased gradient is bounded and well-behaved, producing stable leapfrog trajectories, but the piecewise-smooth loss surface causes Metropolis rejections at the discontinuities. OT Sinkhorn satisfies (2) but can violate (1): the continuous transport makes the loss surface smooth, but differentiating through Sinkhorn can produce gradient spikes. In simple models (e.g. linear Gaussian), systematic resampling works better for HMC because the discontinuities are mild and bounded gradients dominate. In harder models, OT’s continuity becomes essential.

The conditional resampling threshold is a further source of discontinuity affects all filters that resample conditionally. Most particle filters resample only when the effective sample size drops below a threshold, $N_{\text{eff}} < N/2$. During HMC’s leapfrog integration, each step perturbs θ slightly, changing the particle weights and hence the ESS. When the ESS crosses the threshold at some time step t , the filter switches from no resampling to full resampling (or vice versa), and the log-likelihood jumps discontinuously. In

the experiment for cubic sensor model, we observe exactly this phenomenon: shrinking step size for systematic and soft resampling and stable step size for OT Sinkhorn resampling.

For systematic and soft resampling, this jump is large: the resampled particle set (with duplicated high-weight and killed low-weight particles) differs substantially from the unresampled set. The Hamiltonian error $\Delta H \gg 1$, the proposal is rejected, and adaptive step-size tuning shrinks the step size. Since the threshold can be crossed at any scale of perturbation, the step size collapses toward zero.

For OT Sinkhorn resampling, this jump is negligible. When ESS is near the threshold, the weights are close to uniform, and the transport matrix is close to the identity: $T \approx I$, so $x'_i = \sum_j T_{ij} x_j \approx x_i$. Crossing the threshold causes almost no change in the output particles, preserving the Hamiltonian. This is a strong practical argument for fully differentiable resampling: it eliminates the threshold discontinuity by construction.

Unconditional resampling. For systematic and soft resampling, the threshold discontinuity can be removed by resampling unconditionally at every time step. This eliminates the branch point but does *not* eliminate the piecewise-smooth nature of the loss surface — the discrete particle selection still changes as θ varies. For OT resampling, the threshold discontinuity is already negligible, so conditional resampling is acceptable.

Model-dependent choice. The best resampling method depends on the model. For models without sample impoverishment (e.g. linear Gaussian), systematic resampling with detached gradients and unconditional resampling is sufficient and cheapest: the bounded gradient gives stable leapfrog trajectories, and the mild discontinuities do not prevent convergence. For nonlinear models where sample impoverishment is a concern, OT Sinkhorn is preferred: its continuous transport preserves particle diversity and eliminates both sources of discontinuity (threshold and selection).

4.5 Workflow and current status of DPF pipeline

The differentiable filtering module (`src/DF/`) provides three runners, each implementing a different inference strategy:

- **DPFRunner:** HMC on the particle filter’s log-likelihood. A `DifferentiableModel` wraps the state-space model, and a `ParameterHandler` manages bijector transforms between constrained and unconstrained space. Parameters flow as plain tensors: unconstrained $\rightarrow$ bijector $\rightarrow$ `setattr` on the model $\rightarrow$ filter $\rightarrow$ `log_marginal_likelihood_tf`. `tf.GradientTape` differentiates through the entire chain. The particle filter seed is fixed, making the likelihood surface deterministic.
- **PMMHRunner:** Gradient-free. Random-walk Metropolis-Hastings with a fresh seed at each proposal. No `DifferentiableModel` needed — the particle filter is a black box.
- **PGibbsRunner:** Gibbs sampler alternating a θ -step (HMC or MH on the closed-form conditional $p(\theta \mid x_{0:T}, y_{1:T})$) and an x -step (Conditional SMC with ancestor sampling). The θ -step involves no particle filter — just a sum of Gaussian log-densities.

The experiment runner (`run_dpf_experiment.py`) wires everything together via Hydra configs: pick a model, a filter, a runner, and the trainable parameters. We currently have configs for linear-Gaussian, cubic sensor, range-bearing, Kitagawa, and stochastic volatility, crossed with BPF and LEDH invertible, crossed with systematic, soft, and OT resampling.

We started with the linear-Gaussian model as a benchmark. I initially thought this would be easy — we have the Kalman filter as ground truth, so we can check everything. It turned out that systematic resampling is the only method that works cleanly with HMC on this model. Soft and OT resampling introduce extra noise into the gradient that makes the leapfrog integrator less stable, even though the model itself is trivial. This was an early lesson: the resampling method matters more for gradient quality than for filtering quality.

Next I tried range-bearing (2D state, nonlinear observation). Too slow, too many things to debug at once. Not a good model for iterating on the pipeline. So I added the cubic sensor: 1D, mildly nonlinear, fast. Better for debugging, I realized the resampling threshold causing shrinking step size from experiments on this model. Stochastic volatility turned out to be the sweet spot — 1D, nonlinear enough to stress the pipeline, and fast enough to run many experiments.

Along the way, we dealt with several engineering issues. TensorFlow was repeatedly retracing the `GradientTape` computation, causing each HMC step to take minutes instead of seconds. Fixed. Python loops over time steps were slow. Fixed. There are no speed problems anymore. Gradient explosion was a bigger concern: the log-likelihood surface has cliffs near small noise values, and the gradient can blow

up to 10^{10} or produce NaN. We added a diagnostic callback (`on_grad`) that logs the gradient norm at every HMC step, and used gradient clipping to keep the leapfrog integrator stable. The gradients are now finite across all configurations we have tested.

The most recent problem is bad initialization. When the initial guess for θ is far from the truth, the MCMC chain drifts instead of converging. This is not surprising — the likelihood surface is rugged, and HMC with a bad starting point can get stuck in a local mode or wander along a ridge. The plan is to add a MAP warm-start stage: run SGD or Adam for a few hundred steps to find a reasonable θ^* , then start the HMC chain there. Before implementing that, we are testing the pipeline with initial guesses close to the true values to confirm that the inference itself is correct.

LEDH invertible is slower than BPF and has more numerical instability due to the 29-step Jacobian chain. For the DPF pipeline, BPF is the workhorse; LEDH is reserved for models where BPF’s proposal quality is insufficient.

During the early stages, the tests were not structured. Now that the picture is more coherent, we can move to systematic benchmarking.

1. **Comprehensive benchmark.** BPF + OT resampling + stochastic volatility as the primary test case. The goal is to verify that the full pipeline (data generation $\rightarrow$ filtering $\rightarrow$ HMC $\rightarrow$ posterior) produces correct posteriors on a nontrivial model.
2. **Cross-check with Particle Gibbs.** Run PGibbs + MH/HMC on the same model and compare posteriors. This is a different algorithm that avoids differentiating through the filter entirely, so agreement would be strong evidence that both pipelines are correct. However, PGibbs is a separate codebase with its own tuning, so this may digress our attention.
3. **LEDH invertible + look-ahead.** Integrate the bimodal look-ahead version of ledh invertible PF-PF into the DPF pipeline for models like Kitagawa.
4. **MAP warm-start.** Add an SGD/Adam pre-optimization stage to find a good initialization before launching HMC. This should eliminate the parameter drift problem we currently see with bad initial guesses.

4.6 Benchmarking

4.6.1 Benchmarking HMC

For this part, we use Kalman family to test the HMC convergence. If it works here, this is useful baseline for particle filter. We used 100 burn-in steps and 200 sample, with 0.9 target acceptance rate. The blue histogram is the sample distribution and the dark vertical line is the true value corresponding to the data.

Observation: The pipeline worked well for 1-d Gaussian linear and 2-d range bearing model. As for cubic sensor, we need careful tuning to make it better. However the pipeline is still stable.

4.6.2 Benchmarking particle filter and different resampling methods

A key observation is that, if the kalman family works for a model, then differentiable resampling + particle filter combo is not appropriate architecture for Bayesian inference.

Observation: Gradient explosion for any differentiable bootstrap particle filter variation on cubic sensor model. Only thing that stable is the systematic resampling however it will stabilize around the "wrong" minimum. In addition, since anything requiring computing jacobian fails for stochastic volatility model, bpf based method worked really well, (really fast too). However, the posterior is very wide and provide very little constraint, and resampling methods made absolutely no difference.

4.7 Additional notes and practical considerations

Leapfrog divergence. The mechanism by which direct HMC fails is instructive. The leapfrog trajectory starts in a reasonable region, passes through the likelihood cliff near small noise values (where the log-density changes sign), and the enormous gradient launches the state into an extreme parameter region. The Metropolis acceptance correctly rejects such diverged proposals (with acceptance probability $e^{-\Delta H} \approx 0$), but the chain makes no progress. Two remedies exist: gradient clipping, which caps $|\nabla U|$ at a threshold C (a practical hack that preserves detailed balance because the true energy is still used for acceptance), and NUTS (No U-Turn Sampler), which adaptively truncates the trajectory when it starts

doubling back[?]. Both are worth trying for particle filter likelihoods. I did not use NUTS yet due to time cost.

Mass matrix and preconditioning. HMC’s efficiency depends on the mass matrix M matching the posterior geometry. With the default $M = I$, the leapfrog explores all parameter directions at the same rate, which is inefficient when posterior variances differ across parameters. A practical approach is to run a short preliminary chain, estimate the posterior covariance from the samples, and use it as the mass matrix for a longer production run. This problem-specific adjustment is necessary: no off-the-shelf HMC configuration works well for particle filter likelihoods without tuning.

Parameterization. The PMCMC paper uses variances σ^2 with Inverse Gamma priors (conjugate for Gaussian variance), while our setup uses standard deviations σ with a Softplus bijector (log-space HMC). The gradient of $\log p(z|\sigma)$ with respect to $\log \sigma$ involves an extra chain rule factor of σ , which amplifies gradient magnitudes for large σ . Switching to a σ^2 parameterization might smooth the landscape.

Scale We need to increase the scale of experiment, more particles, longer times. But this can only happen for well-tuned pipeline.

A Kalman Family

Notation:

- $\mathbf{m}_t^-$ = *a priori* state estimate (mean of the predicted distribution)
- $\mathbf{m}_t$ = *a posteriori* state estimate (mean of the updated distribution)
- $\mathbf{P}_t^-$ = *a priori* error covariance
- $\mathbf{P}_t$ = *a posteriori* error covariance
- $p(x_k|z_{1:k-1})$ = prior distribution
- $p(x_k|z_{1:k})$ = posterior Distribution

we use the following initial condition,

$$p(x_0) = \mathcal{N}(x_0|m_0, P_0) \quad (40)$$

For Kalman filter, the noise and operators are both Gaussian, so the distributions stay at Gaussian at all times. All we need to do is to keep track of the mean and variance of the prior distribution $p(x_k|y_{1:k})$. In the predict step, the goal is to compute $p(x_t|z_{1:t-1})$ from $p(x_{t-1}|z_{1:t-1})$. Given $p(x_{t-1}|z_{1:t-1}) = \mathcal{N}(x_{t-1}|m_{t-1}, P_{t-1})$, we have:

$$p(x_t|z_{1:t-1}) = \int dx_{t-1} p(x_t|x_{t-1}) p(x_{t-1}|z_{1:t-1}) \quad (41)$$

From the state evolution model:

$$p(x_t|x_{t-1}) = \mathcal{N}(x_t|f x_{t-1}, Q) \quad (42)$$

predicted/evolved mean is,

$$m_t^- = \mathbb{E}[x_t|z_{1:t-1}] \quad (43)$$

$$= f m_{t-1} \quad (44)$$

predicted/evolved variance

$$P_t^- = \text{Var}[x_t|y_{1:t-1}] \quad (45)$$

$$= Q + f^2 P_{t-1} \quad (46)$$

from the predict step, we get,

$$p(x_t|z_{1:t-1}) = \mathcal{N}(x_t|m_t^-, P_t^-) \quad (47)$$

where

$$\boxed{m_t^- = a m_{t-1}, \quad P_t^- = a^2 P_{t-1} + Q} \quad (48)$$

In the update step, we need to incorporate observation z_t using Bayes' rule.

$$p(x_t|z_{1:t}) = \frac{p(z_t|x_t) p(x_t|z_{1:t-1})}{p(z_t|z_{1:t-1})} \quad (49)$$

From the observation model:

$$p(z_t|x_t) = \mathcal{N}(z_t|h x_t, R) \quad (50)$$

We need to compute the product of two Gaussians:

$$p(x_t|z_{1:t}) \propto \mathcal{N}(z_t|h x_t, R) \cdot \mathcal{N}(x_t|m_t^-, P_t^-) \quad (51)$$

$$\propto \exp\left(-\frac{1}{2} \frac{(z_t - h x_t)^2}{R}\right) \exp\left(-\frac{1}{2} \frac{(x_t - m_t^-)^2}{P_t^-}\right) \quad (52)$$

Expanding the exponents:

$$-\frac{1}{2} \left[\frac{(z_t - h x_t)^2}{R} + \frac{(x_t - m_t^-)^2}{P_t^-} \right] = -\frac{1}{2} \left[\frac{z_t^2 - 2 z_t h x_t + h^2 x_t^2}{R} + \frac{x_t^2 - 2 x_t m_t^- + (m_t^-)^2}{P_t^-} \right] \quad (53)$$

Collecting terms in x_t^2 and x_t : $-\frac{1}{2} \left[\left(\frac{h^2}{R} + \frac{1}{P_t^-} \right) x_t^2 - 2 \left(\frac{hz_t}{R} + \frac{m_t^-}{P_t^-} \right) x_t + \text{const} \right]$. This is a Gaussian in x_t . Completing the square, we have $\frac{1}{P_t} = \frac{h^2}{R} + \frac{1}{P_t^-}$. P is the uncertainty so the inverse of P is like precision, by adding information about the measurement $\frac{h^2}{R}$, we get additional precision which is exactly what we are aiming for. When $\frac{h^2}{R} = 0$, namely large noise from observation, we say we can't trust the measurement and we should stick with the dynamics. We see that both mean and variance will receive no update. On the other hand, if $\frac{h^2}{R} \ll \frac{1}{P_t^-}$, then observation will carry more weight and the result from dynamics should be disregarded. The updated variance is

$$P_t = \frac{RP_t^-}{h^2P_t^- + R} \quad (54)$$

For simplicity, we define the **Kalman gain**:

$$K_t = \frac{P_t^- h}{h^2P_t^- + R} \quad (55)$$

Then:

$$P_t = (1 - K_t h) P_t^- \quad (56)$$

After some algebra:

$$m_t = m_t^- + K_t(z_t - hm_t^-) \quad (57)$$

where we denote the innovation $v_t = z_t - hm_t^-$, namely the different between the real observation with the predicted observation from prior mean. In summary, we get,

$$\begin{aligned} K_t &= \frac{P_t^- h}{h^2P_t^- + R} \\ v_t &= y_t - hm_t^- m_t = m_t^- + K_t v_t \\ P_t &= (1 - K_t h) P_t^- \end{aligned}$$

(58)

at this point, we can already see that what filter does is to correct the distribution based on the observed data point. Essentially, we want the mean and covariance from the updated distribution at each time step. For higher dimension, all we have to do is to promote the scalar recursive relation into matrices.

Prediction Step:

$$\hat{\mathbf{x}}_t^- = \hat{\mathbf{F}} \hat{\mathbf{x}}_{t-1}^+ \quad (59)$$

$$\mathbf{P}_t^- = \hat{\mathbf{F}} \mathbf{P}_{t-1}^+ \hat{\mathbf{F}}^T + \mathbf{Q} \quad (60)$$

Correction Step:

$$\hat{\mathbf{K}}_t = \mathbf{P}_t^- \mathbf{H}^T [\mathbf{H} \mathbf{P}_t^- \mathbf{H}^T + \mathbf{R}]^{-1} \quad (\text{Kalman gain}) \quad (61)$$

$$\hat{\mathbf{x}}_t^+ = \hat{\mathbf{x}}_t^- + \hat{\mathbf{K}}_t(\mathbf{z}_t - \mathbf{H} \hat{\mathbf{x}}_t^-) \quad (62)$$

$$\hat{\mathbf{P}}_t^+ = (\mathbf{I} - \hat{\mathbf{K}}_t \mathbf{H}) \mathbf{P}_t^- \quad (63)$$

We know that Kalman filter requires linear models and Gaussian noise. But this is very restrictive. For the cases where the noises are Gaussian but models are non-linear, we have extended Kalman filter (EKF) and unscented Kalman Filter (UKF). The idea for the extension is somehow make approximation so that we can use Kalman filter. EKF linearizes the models via first-order Taylor expansion around current estimate.

$$\mathbf{F}_t = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}_{t-1}^+}, \quad \mathbf{H}_t = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\mathbf{x}=\hat{\mathbf{x}}_t^-} \quad (64)$$

Apply Kalman filter equations using time-varying $\mathbf{F}_t, \mathbf{H}_t$. Evolve using time-dependent operators,

$$\hat{\mathbf{x}}_t^- = f(\hat{\mathbf{x}}_{t-1}^+) \quad (65)$$

$$\mathbf{P}_t^- = \mathbf{F}_t \mathbf{P}_{t-1}^+ \mathbf{F}_t^T + \mathbf{Q} \quad (66)$$

Update using $\mathbf{H}_t$ and predicted measurement $\hat{\mathbf{z}}_t^- = h(\hat{\mathbf{x}}_t^-)$.

$$\hat{\mathbf{K}}_t = \mathbf{P}_t^- \mathbf{H}_t^T [\mathbf{H}_t \mathbf{P}_t^- \mathbf{H}_t^T + \mathbf{R}]^{-1} \quad (\text{Kalman gain}) \quad (67)$$

$$\hat{\mathbf{x}}_t^+ = \hat{\mathbf{x}}_t^- + \mathbf{K}_t (\mathbf{z}_t - \mathbf{H}_t \hat{\mathbf{x}}_t^-) \quad (68)$$

$$\hat{\mathbf{P}}_t^+ = (\mathbf{I} - \mathbf{K}_t \mathbf{H}_t) \mathbf{P}_t^- \quad (69)$$

However, there is strong limitations of this approach, namely, only first-order accurate; can diverge for strong nonlinearity; requires Jacobian computation (may be expensive or analytically intractable); assumes Gaussian noises. As for the UKF, propagate uncertainty through nonlinear functions using **sigma points** (also called unscented points) - deterministically chosen samples that capture mean and covariance. Here we further introduce some definition:

- $\mathbf{P}_{zz,t}^-$ = predicted measurement covariance
- $\mathbf{P}_{xz,t}^-$ = cross-covariance between state and measurement

This is because Kalman gain has a more general form

$$\mathbf{K}_t = \mathbf{P}_{xz,t}^- (\mathbf{P}_{zz,t}^-)^{-1} \quad (70)$$

where

- $\mathbf{P}_{xz,t}^- = \text{Cov}(\mathbf{x}_t - \hat{\mathbf{x}}_t^-, \mathbf{z}_t - \hat{\mathbf{z}}_t^-)$ (state-innovation cross-covariance)
- $\mathbf{P}_{zz,t}^- = \text{Cov}(\mathbf{z}_t - \hat{\mathbf{z}}_t^-)$ (innovation covariance)

This expression is exact for the linear-Gaussian case and approximate (but very effective) for nonlinear cases when the covariances are estimated via linearization (EKF) or sigma points (UKF). Associated with the sigma points are two sets of weights used for computing the mean and covariance after nonlinear transformation:

$$W_0^{(m)} = \frac{\lambda}{n + \lambda} \quad (71)$$

$$W_0^{(c)} = \frac{\lambda}{n + \lambda} + (1 - \alpha^2 + \beta) \quad (72)$$

$$W_i^{(m)} = W_i^{(c)} = \frac{1}{2(n + \lambda)}, \quad i = 1, \dots, 2n \quad (73)$$

where

- $W_i^{(m)}$ are used to recover the transformed **mean**,
- $W_i^{(c)}$ are used to recover the transformed **covariance**,
- β is typically set to 2 (optimal for Gaussian distributions).
- α - Spread Control Parameter, we set it to 10^{-3}
- κ - Secondary Scaling Parameter, we set it to zero.
- $\lambda = \alpha^2(n_x + \kappa) - n_x$

Prediction Step (Time Update):

Generate $2n_x + 1$ sigma points from $\hat{\mathbf{x}}_{t-1}$ and $\mathbf{P}_{t-1}$ (augmented form with process noise is common in literature, but here we show the standard non-augmented version for clarity and consistency with EKF notation):

$$\chi_{t-1}^{(0)} = \hat{\mathbf{x}}_{t-1} \quad (74)$$

$$\chi_{t-1}^{(i)} = \hat{\mathbf{x}}_{t-1} + (\sqrt{(n_x + \lambda) \mathbf{P}_{t-1}})_i \quad i = 1, \dots, n_x \quad (75)$$

$$\chi_{t-1}^{(i)} = \hat{\mathbf{x}}_{t-1} - (\sqrt{(n_x + \lambda) \mathbf{P}_{t-1}})_{i-n_x} \quad i = n_x + 1, \dots, 2n_x \quad (76)$$

Propagate through the nonlinear state transition:

$$\chi_{t|i}^- = f(\chi_{t-1}^{(i)}), \quad i = 0, \dots, 2n_x \quad (77)$$

Compute predicted mean and covariance:

$$\hat{\mathbf{x}}_t^- = \sum_{i=0}^{2n_x} W_i^{(m)} \chi_{t|i}^- \quad (78)$$

$$\mathbf{P}_t^- = \sum_{i=0}^{2n_x} W_i^{(c)} (\chi_{t|i}^- - \hat{\mathbf{x}}_t^-)(\chi_{t|i}^- - \hat{\mathbf{x}}_t^-)^\top \quad (79)$$

Update Step (Measurement Update):

Propagate sigma points through the nonlinear measurement function:

$$\zeta_{t|i}^- = h(\chi_{t|i}^-), \quad i = 0, \dots, 2n_x \quad (80)$$

Compute predicted measurement mean, innovation covariance, and cross-covariance:

$$\hat{\mathbf{z}}_t^- = \sum_{i=0}^{2n_x} W_i^{(m)} \zeta_{t|i}^- \quad (81)$$

$$\mathbf{P}_{zz,t}^- = \sum_{i=0}^{2n_x} W_i^{(c)} (\zeta_{t|i}^- - \hat{\mathbf{z}}_t^-)(\zeta_{t|i}^- - \hat{\mathbf{z}}_t^-)^\top \quad (82)$$

$$\mathbf{P}_{xz,t}^- = \sum_{i=0}^{2n_x} W_i^{(c)} (\chi_{t|i}^- - \hat{\mathbf{x}}_t^-)(\zeta_{t|i}^- - \hat{\mathbf{z}}_t^-)^\top \quad (83)$$

Define the **innovation**:

$$\mathbf{v}_t = \mathbf{z}_t - \hat{\mathbf{z}}_t^- \quad (84)$$

Compute the **Kalman gain**:

$$\mathbf{K}_t = \mathbf{P}_{xz,t}^- (\mathbf{P}_{zz,t}^-)^{-1} \quad (85)$$

Update state mean and covariance:

$$\hat{\mathbf{x}}_t = \hat{\mathbf{x}}_t^- + \mathbf{K}_t \mathbf{v}_t \quad (86)$$

$$\mathbf{P}_t = \mathbf{P}_t^- - \mathbf{K}_t \mathbf{P}_{zz,t}^- \mathbf{K}_t^\top \quad (87)$$

B Particle flow and particle filter

B.1 Particle flow

Previously, we have discussed how after each resampling, the information of the distribution is concentrated on the multiplicity distribution of the particles, and the diversity of the particles have been lost. If we look at the particle methods, it's clearly that we either use weights or multiplicity distribution to mimic distribution. The culprit of the particle degeneracy is the consecutive multiplication of small numbers $p(y_t|x_t^{(i)})$ will make a large number of particles irrelevant after view steps. And the observation was done independently and there is no way to control the value of $p(y_t|x_t^{(i)})$. In [?, ?], the author proposed to use the multiplicity to store the information of the distribution and invented an alternative evolution for the particles such that $p(y_t|x_t^{(i)})$ was never too small. The limitation of these two papers is that to solve the equation, which we will talk about later, they have to assume linear or nearly linearly observation model and dynamics with additive Gaussian noise. The evolution for the particles is complete different equation than underlying dynamics for the ground truth. Later, we will see the methods that generalize the equation for more general noise distribution. Let us first derive the evolution equations. First, we have the prediction step:

$$p(x_t|z_{1:t-1}) = g(x_t) \quad (88)$$

and the "observation" step,

$$p(z_t|x_t) = h(x_t) \quad (89)$$

combine both information, the updated probability for x_t should be,

$$p(x_t|z_{1:t}) \propto g(x_t)h(x_t) \quad (90)$$

The normalization will be handled implicitly by the particles. The difference between the particle flow and the particle filter is how you do the update. Since we are moving the particles directly, we demand,

$$\log p(x, \lambda) = \log g(x) + \lambda \log h(x) \quad (91)$$

we always start with $\lambda = 0$, and end with $\lambda = 1$, therefore we can assure the particle flows to the desired position. By definition, the evolution is driven entirely by the observation model. This is where the limitation of the flow model comes in.

Assuming particles evolve in pseudo-time λ according to the ordinary differential equation:

$$\frac{d\mathbf{x}}{d\lambda} = f(\mathbf{x}, \lambda) \quad (92)$$

where $f(\mathbf{x}, \lambda)$ is the flow field that transports particles from the prior distribution at $\lambda = 0$ to the posterior distribution at $\lambda = 1$. This deterministic evolution of particles leads to a corresponding evolution of the probability density $p(\mathbf{x}, \lambda)$.

The evolution of the probability density is governed by the Fokker-Planck equation with zero diffusion. Starting from the continuity equation for probability conservation, and noting that the flow is deterministic (no diffusion term), we obtain:

$$\frac{\partial p}{\partial \lambda} = -\nabla \cdot (pf) \quad (93)$$

This is the zero-diffusion Fokker-Planck equation, which expresses that the rate of change of probability density equals the negative divergence of the probability flux pf . To work with log-probabilities, we divide both sides by p and apply the product rule for divergence, $\nabla \cdot (pf) = p\nabla \cdot f + (\nabla p) \cdot f$, yielding: Recognizing that $\frac{1}{p} \frac{\partial p}{\partial \lambda} = \frac{\partial \log p}{\partial \lambda}$ and $\frac{\nabla p}{p} = \nabla \log p$, we obtain:

$$\frac{\partial \log p}{\partial \lambda} = -\nabla \cdot f - (\nabla \log p) \cdot f \quad (94)$$

From the log-homotopy definition, we have $\frac{\partial \log p}{\partial \lambda} = \log h(x)$, which leads to the fundamental PDE:

$$\boxed{\log h(x) = -\nabla \cdot f - (\nabla \log p) \cdot f} \quad (95)$$

To solve this equation, we assume an affine flow ansatz:

$$f(x, \lambda) = A(\lambda)x + b(\lambda) \quad (96)$$

where $A(\lambda)$ and $b(\lambda)$ are to be determined. The goal is to find expressions for these functions such that particles following this flow evolve from the prior distribution $g(x)$ at $\lambda = 0$ to the posterior distribution proportional to $g(x)h(x)$ at $\lambda = 1$.

We begin by computing the gradient of the log-homotopy. Using the Gaussian form for both $g(x)$ and $h(x)$, and substituting the explicit forms of the prior and likelihood into the log-homotopy:

$$\log p(x, \lambda) = -\frac{1}{2}(x - \bar{\eta}_0)^T P^{-1}(x - \bar{\eta}_0) - \frac{\lambda}{2}(z - Hx)^T R^{-1}(z - Hx) + \text{const} \quad (97)$$

Taking the gradient with respect to x :

$$\nabla_x \log p(x, \lambda) = -P^{-1}(x - \bar{\eta}_0) + \lambda H^T R^{-1}(z - Hx) \quad (98)$$

$$= -P^{-1}x + P^{-1}\bar{\eta}_0 + \lambda H^T R^{-1}z - \lambda H^T R^{-1}Hx \quad (99)$$

For the affine flow, the divergence is simply the trace of the matrix A :

$$\nabla \cdot f = \nabla \cdot (Ax + b) = \text{Tr}(A) \quad (100)$$

The dot product $(\nabla \log p) \cdot f$ expands as:

$$(\nabla \log p) \cdot f = [-P^{-1}(x - \bar{\eta}_0) + \lambda H^T R^{-1}(z - Hx)]^T [Ax + b] \quad (101)$$

$$= -(x - \bar{\eta}_0)^T P^{-1}Ax - (x - \bar{\eta}_0)^T P^{-1}b \quad (102)$$

$$+ \lambda(z - Hx)^T R^{-1}HAx + \lambda(z - Hx)^T R^{-1}Hb \quad (103)$$

The log-likelihood for the linear-Gaussian observation model can be expanded as:

$$\log h(x) = -\frac{1}{2}(z - Hx)^T R^{-1}(z - Hx) + \text{const} \quad (104)$$

$$= -\frac{1}{2}z^T R^{-1}z + z^T R^{-1}Hx - \frac{1}{2}x^T H^T R^{-1}Hx + \text{const} \quad (105)$$

A key insight for the EDH flow is that, under Gaussian assumptions, the distribution at any pseudo-time λ remains Gaussian. Specifically, the posterior at λ is:

$$p(x, \lambda) \sim \mathcal{N}(\bar{\eta}_\lambda, P_\lambda) \quad (106)$$

where the precision matrix and mean evolve according to:

$$P_\lambda^{-1} = P^{-1} + \lambda H^T R^{-1}H \quad (107)$$

$$\bar{\eta}_\lambda = P_\lambda[P^{-1}\bar{\eta}_0 + \lambda H^T R^{-1}z] \quad (108)$$

This allows us to express the gradient of the log-probability in a compact form:

$$\nabla \log p(x, \lambda) = -P_\lambda^{-1}(x - \bar{\eta}_\lambda) \quad (109)$$

Substituting this into the fundamental PDE, we obtain an alternative formulation:

$$\log h(x) = -\text{Tr}(A) + (x - \bar{\eta}_\lambda)^T P_\lambda^{-1}(Ax + b) \quad (110)$$

A more direct approach to determining $A(\lambda)$ and $b(\lambda)$ comes from requiring that particles starting from a Gaussian distribution remain Gaussian under the flow. For an affine flow $f = Ax + b$, the evolution of the mean and covariance must satisfy:

$$\frac{d\bar{\eta}_\lambda}{d\lambda} = A\bar{\eta}_\lambda + b, \quad \frac{dP_\lambda}{d\lambda} = AP_\lambda + P_\lambda A^T \quad (111)$$

From Kalman filter theory, we know that the precision matrix evolves as:

$$\frac{dP_\lambda^{-1}}{d\lambda} = H^T R^{-1}H \quad (112)$$

Using the identity $\frac{dP_\lambda}{d\lambda} = -P_\lambda \frac{dP_\lambda^{-1}}{d\lambda} P_\lambda$, we find:

$$\frac{dP_\lambda}{d\lambda} = -P_\lambda H^T R^{-1} H P_\lambda \quad (113)$$

Matching this with the covariance evolution equation $AP_\lambda + P_\lambda A^T = -P_\lambda H^T R^{-1} H P_\lambda$, and solving the resulting Riccati-type equation with boundary condition $P_0 = P$, we obtain:

$$A(\lambda) = -\frac{1}{2} P_\lambda H^T R^{-1} H = -\frac{1}{2} P H^T (\lambda H P H^T + R)^{-1} H \quad (114)$$

For the mean evolution, we require that $\frac{d\bar{\eta}_\lambda}{d\lambda} = H^T R^{-1}(z - H\bar{\eta}_\lambda)$ equals $A\bar{\eta}_\lambda + b$. Solving for b :

$$b(\lambda) = P_\lambda H^T R^{-1} z - A\bar{\eta}_\lambda \quad (115)$$

After algebraic manipulation, this simplifies to:

$$b(\lambda) = (I + 2\lambda A)[(I + \lambda A)P H^T R^{-1} z + A\bar{\eta}_0] \quad (116)$$

The EDH flow equations are derived by requiring that Gaussian distributions remain Gaussian under the flow, that the flow satisfies the Fokker-Planck equation, and that the boundary conditions $p(x, 0) = g(x)$ and $p(x, 1) \propto g(x)h(x)$ are satisfied. The resulting solution provides an **exact** method for transporting particles from the prior to the posterior in the linear-Gaussian case, avoiding the weight degeneracy issues that plague standard particle filters. As for the LEDH, the difference is that each particle has their own equation. That is because the expansion of the model are taken at their individual location instead of the average position.

B.2 Particle flow filter (kernel method)

Our problem is the same as particle filter particle flow, move the particles to the desired location, without touching the weights, so that we get the target probability distribution $p(x_t|y_{1:t}) \propto p(y_t|x_t)p(x_t|y_{1:t-1})$.

B.2.1 Optimal Transport via Gradient Flow

Define target posterior $\pi(x) := p(x_t|y_{1:t})$ and intermediate density $q_\lambda(x)$ at pseudo-time $\lambda \in [0, 1]$. With $q_0 = p(x_t|y_{1:t-1})$ (prior), $q_1 = \pi$ (posterior).

$$\frac{dx_\lambda}{d\lambda} = v_\lambda(x_\lambda) \quad (117)$$

Choose velocity v_λ to minimize KL divergence as fast as possible:

$$D_{KL}(q_\lambda \parallel \pi) = \int q_\lambda(x) \log \frac{q_\lambda(x)}{\pi(x)} dx \quad (118)$$

we need to formulate the above equation into an optimization problem. But the intuition of this method is clear, you want to evolve the point along a "straight" direction to the desire point.

B.2.2 Deriving Optimal Velocity via Kernel Restriction

Under velocity field v , density evolves via Liouville equation:

$$\frac{\partial q_\lambda}{\partial \lambda} = -\nabla_x \cdot (q_\lambda v) \quad (119)$$

Time derivative of D_{KL} :

$$\frac{d}{d\lambda} D_{KL} = \int \frac{\partial q_\lambda}{\partial \lambda} [\log q_\lambda(x) - \log \pi(x) + 1] dx \quad (120)$$

$$= - \int \nabla_x \cdot (q_\lambda v) [\log q_\lambda(x) - \log \pi(x) + 1] dx \quad (121)$$

Using $\nabla_x q_\lambda = q_\lambda \nabla_x \log q_\lambda$ and integrating by parts:

$$\frac{d}{d\lambda} D_{KL} = - \int q_\lambda [\nabla_x \log \pi(x) \cdot v + \nabla_x \cdot v] dx \quad (122)$$

The derivative with respect to the evolution parameter gives the vector field along the trajectory. To make things computable, we restrict v to functions generated by kernel $K(x, x')$. Any such function has form:

$$v(x) = \int \alpha(x') K(x', x) dx' \quad (123)$$

for some coefficient function $\alpha(x')$. Substitute this into directional derivative.

$$\frac{d}{d\lambda} D_{KL} = - \int q_\lambda(x) \left[\nabla_x \log \pi(x) \cdot \int \alpha(x') K(x', x) dx' + \nabla_x \cdot \int \alpha(x') K(x', x) dx' \right] dx \quad (124)$$

Interchange integrals and use integration by parts:

$$\frac{d}{d\lambda} D_{KL} = \int \alpha(x) \cdot \phi(x) dx \quad (125)$$

where

$$\phi(x) = - \int q_\lambda(x') [K(x, x') \nabla_{x'} \log \pi(x') + \nabla_{x'} K(x, x')] dx' \quad (126)$$

We have $\frac{d}{d\lambda} D_{KL} = \langle \alpha, \phi \rangle$ (inner product in function space).

To minimize this quantity (make KL decrease fastest), choose α antiparallel to ϕ . By Cauchy-Schwarz, the minimum is achieved when:

$$\alpha(x) = -\phi(x) \quad (127)$$

Therefore, optimal velocity field:

$$v_\lambda(x) = \int \alpha(x') K(x', x) dx' \quad (128)$$

$$= - \int \phi(x') K(x', x) dx' \quad (129)$$

By the reproducing property of the kernel, this simplifies to:

$$v_\lambda(x) = \mathbb{E}_{x' \sim q_\lambda} [K(x, x') \nabla_{x'} \log \pi(x') + \nabla_{x'} K(x, x')] \quad (130)$$

Monte Carlo approximation with particles $\{x_\lambda^{(i)}\}_{i=1}^{N_p} \sim q_\lambda$:

$$v_\lambda(x) \approx \frac{1}{N_p} \sum_{i=1}^{N_p} [K(x_\lambda^{(i)}, x) \nabla \log \pi(x_\lambda^{(i)}) + \nabla K(x_\lambda^{(i)}, x)] \quad (131)$$

Posterior gradient:

$$\nabla \log \pi(x) = \nabla \log p(y_t | x) + \nabla \log p(x | y_{1:t-1}) \quad (132)$$

First term = likelihood gradient (known). Second term = prior gradient (approximated from ensemble).

Update rule:

$$x_{\lambda+\epsilon}^{(j)} = x_\lambda^{(j)} + \frac{\epsilon}{N_p} \sum_{i=1}^{N_p} [K(x_\lambda^{(i)}, x_\lambda^{(j)}) \nabla \log \pi(x_\lambda^{(i)}) + \nabla K(x_\lambda^{(i)}, x_\lambda^{(j)})] \quad (133)$$

All particles maintain equal weight $w^{(i)} = 1/N_p$ throughout. No resampling needed.

Generality: Works for any differentiable likelihood $p(y_t | x_t)$ - not limited to Gaussian noise.

B.2.3 Scalar Kernel

Standard choice: isotropic Gaussian

$$K(x, x') = \exp \left(-\frac{1}{2\alpha\sigma^2} \|x - x'\|^2 \right), \quad \alpha \sim n_x \quad (134)$$

Gradient: $\nabla_x K(x, x') = -\frac{1}{\alpha\sigma^2} (x - x') K(x, x')$

Single scalar $K(x, x')$ weights all components identically.

B.2.4 Matrix-Valued Kernel

Measure distance independently per component:

$$\mathbf{K}(x, x') = \text{diag} \left[K^{(1)}(x, x'), \dots, K^{(n_x)}(x, x') \right] \quad (135)$$

where

$$K^{(d)}(x, x') = \exp \left(-\frac{(x^{(d)} - x'^{(d)})^2}{2\alpha(\sigma^{(d)})^2} \right), \quad \alpha = \frac{1}{N_p} \quad (136)$$

Component-wise repulsion:

$$[\nabla_x \cdot \mathbf{K}(x, x')]^{(d)} = -\frac{x^{(d)} - x'^{(d)}}{\alpha(\sigma^{(d)})^2} K^{(d)}(x, x') \quad (137)$$

Observed dimensions maintain diversity regardless of unobserved dimension spread.

C MCMC Diagnostics

This appendix details the four diagnostic quantities reported after every HMC/NUTS run.

C.1 Acceptance Rate

At each HMC step a proposal q^* is drawn by integrating Hamilton’s equations for L leapfrog steps from the current state q . The joint state is (q, p) where $p \sim \mathcal{N}(0, I)$ is the auxiliary momentum. The *Hamiltonian* is

$$H(q, p) = -\log \pi(q) + \frac{1}{2} \|p\|^2, \quad (138)$$

where $\pi(q)$ is the target density (here the posterior). The Metropolis acceptance probability is

$$\alpha = \min(1, e^{-\Delta H}), \quad \Delta H = H(q^*, p^*) - H(q, p). \quad (139)$$

If the leapfrog integrator were exact, $\Delta H = 0$ and every proposal would be accepted. Numerical discretisation error causes $\Delta H \neq 0$, so some proposals are rejected.

Example. Suppose $\Delta H = 0.5$. Then $\alpha = e^{-0.5} \approx 0.607$, i.e. the proposal is accepted with probability 61%. If $\Delta H < 0$ (proposal has lower energy / higher probability), the proposal is always accepted.

Target acceptance rate. For standard HMC, the theoretically optimal acceptance rate in high dimensions is approximately 0.65 [?]. For NUTS and problems with smoother posteriors, a higher target of 0.80–0.90 is common. Acceptance rates near 100% indicate the step size is too small: proposals barely move and consecutive samples are highly correlated.

C.2 Step Size and Dual-Averaging Adaptation

The leapfrog step size ε controls how far each sub-step moves in parameter space. A total HMC trajectory covers approximately $L\varepsilon$ in unconstrained space. Too large ε causes large ΔH and frequent rejection; too small ε gives near-certain acceptance but slow exploration.

The step size is adapted during burn-in using *dual averaging* (Hoffman & Gelman, 2014 [?]), which is the same algorithm used in our custom HMC implementation (`hmc_runner.py`). Let δ be the target acceptance probability and α_m the actual acceptance probability at step m . The update equations are:

$$\bar{H}_m = \left(1 - \frac{1}{m + t_0}\right) \bar{H}_{m-1} + \frac{1}{m + t_0} (\delta - \alpha_m), \quad (140)$$

$$\log \varepsilon_m = \mu - \frac{\sqrt{m}}{\gamma} \bar{H}_m, \quad (141)$$

$$\log \bar{\varepsilon}_m = m^{-\kappa} \log \varepsilon_m + (1 - m^{-\kappa}) \log \bar{\varepsilon}_{m-1}. \quad (142)$$

Here $\mu = \log(10\varepsilon_0)$ is a free target (set to $10\times$ the initial step size), $\gamma = 0.05$ is a shrinkage factor, $t_0 = 10$ stabilises early iterations, and $\kappa = 0.75$ controls the long-run averaging weight. The *running average* $\bar{\varepsilon}_m$ is used as the final step size after adaptation ends; this is more stable than the instantaneous ε_m . In our code the adaptation runs for the first 80% of burn-in steps.

Intuition. When $\alpha_m < \delta$ the chain is rejecting too often, so $\bar{H}_m$ increases and Eq. (141) decreases ε . When $\alpha_m > \delta$ the opposite happens. The exponential averaging in Eq. (142) smooths out this feedback.

C.3 Effective Sample Size

Because consecutive MCMC samples are correlated, N correlated samples carry less information than N independent ones. The *effective sample size* (ESS) quantifies how many independent samples the chain is equivalent to:

$$\text{ESS} = \frac{N}{1 + 2 \sum_{k=1}^{\infty} \rho_k}, \quad (143)$$

where $\rho_k = \text{Corr}(\theta_t, \theta_{t+k})$ is the lag- k autocorrelation of the chain. In practice the infinite sum is truncated where the autocorrelations become negligible. Our implementation calls `tfp.mcmc.effective_sample_size`, which estimates the sum via FFT-based autocorrelation.

Demonstration: AR(1) chain. Suppose $\theta_t = \phi \theta_{t-1} + \varepsilon_t$ with $|\phi| < 1$. Then $\rho_k = \phi^k$, and the geometric series gives

$$\text{ESS} = N \frac{1 - \phi}{1 + \phi}. \quad (144)$$

Autocorrelation ϕ	$N = 200$ samples	ESS
0.0	200	200 (independent)
0.5	200	67
0.9	200	≈ 11 (poor mixing)
0.99	200	≈ 1 (stuck)

Reliability. With only $N = 200$ post-burn-in samples the autocorrelation estimator is itself noisy. The FFT-based estimator can underestimate the true autocorrelation, causing ESS to saturate at N even when mixing is imperfect. ESS estimates require at least several hundred samples to be trustworthy.

C.4 $\hat{R}$: Potential Scale Reduction Factor

The $\hat{R}$ statistic (Gelman & Rubin, 1992) compares *within-chain* variance to *between-chain* variance to detect non-convergence. Since we run a single chain, we follow the common practice of splitting the N post-burn-in samples into two halves of length $n = N/2$ each, treating them as independent chains ($m = 2$).

Let $\bar{\theta}_j$ and s_j^2 be the mean and variance of chain j . Define

$$W = \frac{1}{m} \sum_{j=1}^m s_j^2 \quad (\text{within-chain variance}), \quad (145)$$

$$B = \frac{n}{m-1} \sum_{j=1}^m (\bar{\theta}_j - \bar{\bar{\theta}})^2 \quad (\text{between-chain variance}), \quad (146)$$

$$\hat{V} = \frac{n-1}{n} W + \frac{1}{n} B \quad (\text{pooled variance estimate}). \quad (147)$$

Then

$$\hat{R} = \sqrt{\frac{\hat{V}}{W}}. \quad (148)$$

Interpretation. If the chain has converged, both halves should look like draws from the same stationary distribution, so $B \approx 0$ and $\hat{V} \approx W$, giving $\hat{R} \approx 1$. A value $\hat{R} > 1.1$ is the standard threshold for flagging non-convergence.

Why $\hat{R}$ can be less than 1. Mathematically, $\hat{R} < 1$ requires $\hat{V} < W$, which happens when B is very small (the two half-chains have nearly identical means) but W is inflated by within-chain fluctuations. With only $n = 100$ samples per half-chain, both W and B are estimated with high uncertainty, and values like $\hat{R} = 0.83$ are consistent with a correctly converging chain that simply has too few samples for reliable estimation. At least $n \geq 200$ per chain ($N \geq 400$ total) is recommended before trusting $\hat{R}$.